A Project Report on

Quiz Management System

Submitted for the partial fulfillment of
Project in Bachelor of Computer
Application
(Semester – V)

-: Submitted To:-

Department of Computer Science,
Harivandana College, Rajkot.

-: Affiliated to :-

Saurashtra University, Rajkot

-: Submitted By :-
Tushar Parmar
(Enroll no:24CS002UG00220)

-: Under the Guidance of :-
Dr. Ashwin Rathod
(Principal, Harivandana College)
Prof. Bhavesh Chavda
(Assistant Professor & Project in charge)

CERTIFICATE

HARIVANDANA
EDUCATION TRUST

Harivandana
College
Affiliated to Saurashtra University

Date: 16-09-2026

CERTIFICATE

This is to certify that the project report entitled **QUIZ MANAGEMENT SYSTEM** submitted to **Harivandana College, Rajkot,** in partial fulfilment of the requirements for the Current Semester **B.C.A SEM-5 - 2026**, work carried out by **PARMAR TUSHAR RANCHHODBHAI.**

The project process is carried out during the present semester in computer laboratories under the supervision of a guide preferably from the college. During this semester, the student has gone through several theoretical reports such as SDLC, Database Management System, S/W Engineering etc. The matter embodied in this project is satisfactorily done by the student for the fulfilment of the required semester.

Principal
&
Head of Department
Dr.Ashwin Rathod

Internal Guide
Prof. Bhavesh Chavda

Campus:
Nr. Saurashtra University
Campus, Munjka,
Rajkot - 360005

Phone : 0281 2575594
Mobile : 99781 55555
Email : harivandana@yahoo.co.in
Web : www.harivandanacollege.org

ACKNOWLEDGEMENT

While preparing and presenting this project, I understood the difference between theoretical and practical aspect, without taking help of other people it is not possible to complete this project. So we have opportunity to say thank them all who have Help us directly or indirectly to make the project successful like internal & external guide.

Firstly, I would like to thank You **Prof. Bhavesh Chavda** for guide in this whole project. Prolonged interest in my work and excellence guidance. He has been a constant source of motivation to me by providing us with suitable performance.

For giving us throughout the project and for the constant support of them and help in design and implementation of this project. Moreover, I would especially like to thank my friends and teachers and last but not the least I am great full to my parents for directly support and unconditional help which made my project a real success.

TABLE OF CONTENTS

No.	Subject Title	Page No.
1	**Introduction and Objectives**	1
	1.1 Project Summary	2
	1.2 Purpose	3
	1.3 Scope	3
	1.4 Objectives	4
	1.5 Target Users	5
	1.6 Advantages	6
2	**Literature Review**	7
	2.1 Web Architecture and Client-Server Model	8
	2.2 PHP	8
	2.3 MySQL	9
	2.4 Apache and XAMPP	10
	2.5 HTML5 and CSS3	10
	2.6 JavaScript	11
	2.7 AdminLTE & Bootstrap	12
3	**System Analysis**	13
	3.1 Problem Definition	14
	3.2 Feasibility Study	14
	3.3 Requirements and Specifications	15
	3.4 Functional Requirements	16
	3.5 Non-Functional Requirements	18
4	**Project Management**	19
	4.1 Project Planning and SDLC Model	20
	4.2 SDLC Phases	20
	4.3 Project Schedule	21
	4.4 Gantt Chart	22
5	**System Design**	24
	5.1 System Architecture	25

	5.2 Use Case Diagram	26
	5.3 Activity Diagram	27
	5.4 ER Diagram	28
	5.5 Relational Schema	29
	5.6 Context Diagram	30
	5.7 System Flowchart	31
	5.8 Data Flow Diagram	32
	5.8.1 DFD Level 0	33
	5.8.2 DFD Level 1	34
6	**Data Dictionary**	35
	6.1 Database Overview	36
	6.2 Department Table (dept)	37
	6.3 Staff Table (staff)	38
	6.4 Student Table (student)	40
	6.5 Quiz Table (quiz)	42
	6.6 Questions Table (questions)	43
	6.7 Score Table (score)	45
	6.8 Stored Procedures and Triggers	47
	6.9 Referential Integrity and Normalization	47
7	**Implementation Screenshots**	48
	7.1 Staff / Admin Interface	49
	7.1.1 Staff / Admin Login Interface	49
	7.1.2 Staff / Administrator Registration Page	50
	7.1.3 Staff / Admin Dashboard	52
	7.1.4 Quiz Management Table	54
	7.1.5 Create New Quiz	55
	7.1.6 Add Questions to Quiz	56
	7.1.7 Quiz Questions	59
	7.1.8 Student Leaderboard (Staff View)	61
	7.1.9 Admin Profile	62
	7.1.10 Edit Administrator Profile and Website Logo	63
	7.2 Client / Student Interface	65

	7.2.1 Student Registration Form	65
	7.2.2 Student Dashboard	66
	7.2.3 Student Scoreboard	67
	7.2.4 Student Leaderboard	68
	7.2.5 Student Profile	69
	7.2.6 Student Quiz Attempt Page	71
8	**Testing**	74
	8.1 Testing Overview	75
	8.2 Testing Methodology	75
	8.3 Unit Testing	75
	8.4 Integration Testing	76
	8.5 System Testing	76
	8.6 Test Cases and Results	77
9	**Conclusion & Future Scope**	79
	9.1 Conclusion and Future Scope	80
	9.2 Achievements	80
	9.3 Limitations	81
	9.4 Future Scope	81
10	**Reference and Bibliography**	82
	10.1 Books and Academic References	83
	10.2 Official Documentation	83
	10.3 Online Resources	84

1.Introduction and Objectives

1.1 Project Summary

The Quiz Management System is a web application designed to support online quiz creation, participation, and evaluation in an academic environment. It provides separate interfaces for staff members and students. Staff members can create quizzes, add multiple-choice questions, manage quiz information, and review student results. Students can register, sign in, view available quizzes, answer questions, submit their responses, and review their scores.

The system is developed using PHP, MySQL, HTML5, CSS3, and JavaScript. Apache and MySQL are run through XAMPP during local development, while Bootstrap and AdminLTE provide reusable interface and dashboard components. Quiz information, user accounts, questions, and score records are stored in the MySQL database. When a student submits a quiz, the application evaluates the answers and records the result.

1.2 Purpose

The purpose of this project is to provide a centralized platform for conducting objective-type quizzes. In a manual process, staff may need to prepare and distribute questions, collect answers, check each response, calculate scores, and maintain result records. This can take considerable time and make it harder to keep records organized.

1.3 Scope

The system focuses on creating, conducting, and evaluating multiple-choice quizzes. It supports two main user roles:

- **Staff/Admin:** Can register and sign in, manage profile information, create quizzes, add questions and answer options, select correct answers, and manage quiz content.
- **Student:** Can register and sign in, view available quizzes, attempt questions, submit answers, review scores, view a leaderboard, and access profile information.

1.4 Objectives

The main objective is to develop a web-based system that supports the management and evaluation of academic quizzes. The specific objectives are:

1. Provide registration and sign-in functionality for students and staff.
2. Provide separate interfaces based on user roles.
3. Allow staff to create quizzes and manage their questions.
4. Allow students to browse and attempt available quizzes.
5. Evaluate student responses and calculate scores automatically.
6. Store quiz attempts and results in the MySQL database.
7. Provide students with access to their score history and leaderboard information.
8. Reduce the paperwork and manual effort involved in quiz administration and evaluation.

1.5 Target Users

The primary users are **staff members** and **students** in an educational institution. Staff use the system to prepare quiz content, maintain quizzes, and review student performance. Students use it to access quizzes, submit answers, and view their results.

Academic departments or teaching teams may also use the system to support classroom assessments and monitor quiz participation. Each user accesses the functions intended for their role through the corresponding interface.

1.6 Advantages

The **Quiz Management System** provides several advantages over traditional manual quiz systems:

- **Automated evaluation:** Student answers are evaluated automatically, reducing the time required for checking and calculating scores.
- **Improved accuracy:** Automatic score calculation minimizes human errors in evaluation and result preparation.
- **Centralized data management:** Student details, staff records, quizzes, questions, and scores are stored in a single MySQL database.
- **Easy quiz creation:** Staff members can create, update, view, and delete quizzes through an organized interface.
- **Immediate results:** Students can view their scores shortly after submitting a quiz.
- **User-based access:** Separate student and staff/admin interfaces ensure that users can access only the functions related to their roles.
- **Reduced paperwork:** The system minimizes the use of printed question papers, answer sheets, and manually maintained records.

2.Literature Review

2.1 Web Architecture and Client-Server Model

The Quiz Management System follows a web-based client-server model. The client is the user's web browser, through which students and staff access the application. The server receives browser requests, processes them, and returns the requested pages or results.

In this project, Apache handles requests for the application, PHP performs server-side processing, and MySQL stores and retrieves system information. For example, when a student opens a quiz, the browser requests the quiz page, PHP retrieves the quiz and questions from MySQL, and the server returns the generated page. When the student submits answers, PHP evaluates them and saves the result in the database.

2.2 PHP

PHP is the server-side programming language used to implement the application's main functions. It processes form submissions, connects to the database, manages login sessions, retrieves quiz information, and generates dynamic web pages.

In this project, PHP supports student and staff registration and authentication. It also handles quiz creation, question management, answer evaluation, score storage, profile display, and leaderboard retrieval. Session variables are used to identify the logged-in user and determine which role-specific pages they may access.

2.3 MySQL

MySQL is the relational database management system used to store the application's persistent data. It organizes records into related tables, allowing information to be retrieved and maintained through SQL queries.

The project database contains tables for departments, staff, students, quizzes, questions, and scores. Quiz identifiers connect quizzes with their questions and score records. Email fields associate staff members with created quizzes and students with their results. This structure allows the application to retrieve quiz content, record attempts, display scores, and produce leaderboard information.

2.4 Apache and XAMPP

Apache is the web server used to serve the Quiz Management System to users' browsers. It receives page requests and works with PHP to process application files before returning the generated output.

XAMPP provides a local development environment that includes Apache, PHP, and MySQL. In this project, Apache serves the application, PHP executes its server-side code, and MySQL stores the data. The project can be run locally by placing its files in the XAMPP htdocs directory and starting the required services from the XAMPP control panel.

2.5 HTML5 and CSS3

HTML5 provides the structure of the application's web pages. It is used to create page sections, headings, forms, labels, buttons, links, tables, and other interface elements. The login, registration, quiz-taking, dashboard, and profile pages use HTML structures to present information and collect user input.

CSS3 controls the visual presentation of those elements. It is used to apply colors, spacing, typography, borders,

layouts, and responsive styling. The student-facing pages combine custom styles with reusable interface components to present quizzes, scores, and profile information consistently.

2.6 JavaScript

JavaScript adds client-side interaction to the application. It can respond to user actions in the browser without requiring a complete page reload for every interface change.

In the project, JavaScript supports actions such as switching between student and staff registration forms, changing the selected login role, confirming potentially important actions, and previewing uploaded images. These features make forms and pages more interactive and help users understand the effects of their actions.

2.7 AdminLTE & Bootstrap

Bootstrap is a front-end framework that provides reusable styling and layout components, including responsive grids, forms, buttons, tables, alerts, and navigation elements. These components help maintain a consistent design across different screen sizes.

AdminLTE is a dashboard template built using Bootstrap and related web technologies. It provides an administrative layout with a navigation sidebar, page header, dashboard cards, tables, and other interface components. In this project, AdminLTE is used mainly for the staff/admin interface, including the dashboard, quiz-management table, question-management pages, leaderboard, and profile settings.

3.System Analysis

3.1 Problem Definition

Managing quizzes manually involves preparing questions, distributing them, collecting answers, marking responses, calculating scores, and maintaining result records. These tasks can take considerable time and may lead to calculation errors or disorganized records.

The Quiz Management System addresses this problem by providing a web application where staff can create quizzes and questions, and students can attempt quizzes online. The system evaluates objective answers and stores scores in a central database, allowing students to review results and staff to view performance information.

3.2 Feasibility Study

A feasibility study helps determine whether the project can be developed and used with available resources.

- **Technical feasibility:** The project uses PHP, MySQL, Apache, HTML5, CSS3, JavaScript, Bootstrap, and AdminLTE. These technologies work together in the XAMPP development environment and are suitable for a browser-based academic application.

- **Economic feasibility:** The project can be developed and tested using open-source software. It does not require paid development tools or dedicated hardware beyond a computer capable of running XAMPP.
- **Operational feasibility:** The system provides separate interfaces for students and staff. Staff manage quiz content and review results; students browse quizzes, submit answers, and view their scores.
- **Schedule feasibility:** The system can be developed in stages, including requirements analysis, database and interface design, implementation, testing, and documentation, making it appropriate for an academic project schedule.

3.3 Requirements and Specifications

The system requires a server environment that can run PHP and MySQL. During local development, these services are provided through XAMPP, with Apache serving the application pages. Users access the system through a web browser.

The application supports two roles: **staff** and **student**. Staff members can create and manage quizzes and

questions, and view quiz-related student results. Students can register, sign in, view available quizzes, answer questions, submit attempts, and review their results. The database stores department, staff, student, quiz, question, and score information.

The system accepts user details and quiz answers through web forms. It retrieves or updates the related database records and returns pages, messages, quiz questions, scores, or leaderboard information to the user.

3.4 Functional Requirements

Functional requirements describe the operations provided by the application.

User functions

- Students and staff can register and sign in.
- Users are directed to the interface for their role after successful login.
- Users can sign out of the application.
- Staff and students can view their profile information; profile updates are available through the relevant interface.

Staff functions

- Create a quiz and add multiple-choice questions.
- Select the correct answer from the question options.
- View quiz details and configured questions.
- Edit quiz information and questions.
- Delete quizzes or individual questions.
- Review student scores and leaderboard information for their quizzes.

Student functions

- View the available quizzes and their question counts.
- Start a quiz and select an answer for each question.
- Submit answers for automatic evaluation.
- View the score, total marks, and performance remark.
- Review previous quiz attempts and leaderboard rankings.
- View profile information and quiz activity statistics.

Data functions

- Store and retrieve user, department, quiz, question, and score records.
- Associate questions with their quiz and score records with the corresponding student and quiz.
- Record quiz attempts and use stored results to display score history and leaderboard data.

3.5 Non-Functional Requirements

Non-functional requirements describe qualities and operating conditions expected of the system.

- **Usability:** Pages and forms should be understandable to students and staff.
- **Performance:** Common operations such as opening a dashboard, loading quiz questions, and submitting answers should complete within a reasonable time for the available server.
- **Reliability:** Quiz, question, and score information should be stored and retrieved consistently.
- **Access control:** Staff and student pages should be available to the appropriate logged-in user role.

4.Project Management

4.1 Project Planning and SDLC Model

The project was organized using the **Software Development Life Cycle (SDLC)** to divide development into manageable stages. The work began with identifying the requirements for student and staff users, followed by designing the database and interfaces. The system was then implemented using PHP and MySQL, tested in the XAMPP environment, and documented.

An iterative approach was suitable because the system consists of connected features—authentication, quiz management, question management, quiz attempts, and score reporting. Individual features could be developed and checked before integrating them into the complete application.

4.2 SDLC Phases

1. **Planning:** Define the project purpose, objectives, scope, and required technologies.
2. **Requirement analysis:** Identify the functions required by students and staff.
3. **System design:** Plan the database, application structure, user interfaces, and data flows.

4. **Implementation:** Develop the application pages, PHP processing, and MySQL database operations.
5. **Testing:** Check registration, login, quiz creation, question entry, answer evaluation, and score display.
6. **Deployment:** Run the application using Apache and MySQL through XAMPP.
7. **Maintenance:** Correct defects and make future enhancements as requirements change.

4.3 Project Schedule

The development of the Quiz Management System was organized according to a planned project schedule. The initial stage focused on identifying the limitations of manual quiz systems and collecting the requirements of students and staff/admin users. This was followed by system analysis, database planning, interface design, and selection of the required technologies. The implementation stage included developing the authentication module, registration functionality, staff/admin dashboard, student dashboard, quiz and question management, quiz-taking interface, automatic evaluation, score storage, and leaderboard features. After implementation, the individual modules were integrated

and tested to identify and correct errors. The final stage involved system deployment in the XAMPP environment, preparation of screenshots and test results, and completion of the project report. Organizing the work into these stages helped ensure that the project progressed systematically from initial planning to final documentation.

4.4 Gantt Chart

The Gantt chart presents the planned timeline for developing the **Quiz Management System**. It shows the sequence and approximate duration of the major project activities, including requirement analysis, feasibility study, system design, database design, interface development, module implementation, integration, testing, deployment, and documentation. The activities were arranged in a logical order, although some tasks could be performed concurrently. Requirement analysis and system design were completed before implementation began, while database development and interface development progressed alongside application coding. Testing was carried out after the main modules were implemented and integrated. The Gantt chart helped monitor project progress, allocate sufficient time to each

activity, identify dependencies between tasks, and ensure that the system and its documentation were completed within the planned academic schedule.

Phase	Milestone Description	Duration	Start Week	End Week	Status
Phase 1	Literature Review & Feasibility Study	2 Weeks	Week 1	Week 2	Completed
Phase 2	Requirement Gathering & SRS Formulation	2 Weeks	Week 3	Week 4	Completed
Phase 3	Architectural & Normalized DB Design	2 Weeks	Week 5	Week 6	Completed
Phase 4	Triggers, Procedures & Core Backend	3 Weeks	Week 7	Week 9	Completed
Phase 5	AdminLTE 3 & Student UI Implementation	3 Weeks	Week 10	Week 12	Completed

5.System Design

5.1 System Architecture

The system uses a web-based client-server architecture. Students and staff access it through a browser. Apache receives the requests, PHP processes them, and MySQL stores and retrieves the application data. The result—such as a dashboard, quiz, or score—is returned to the browser.

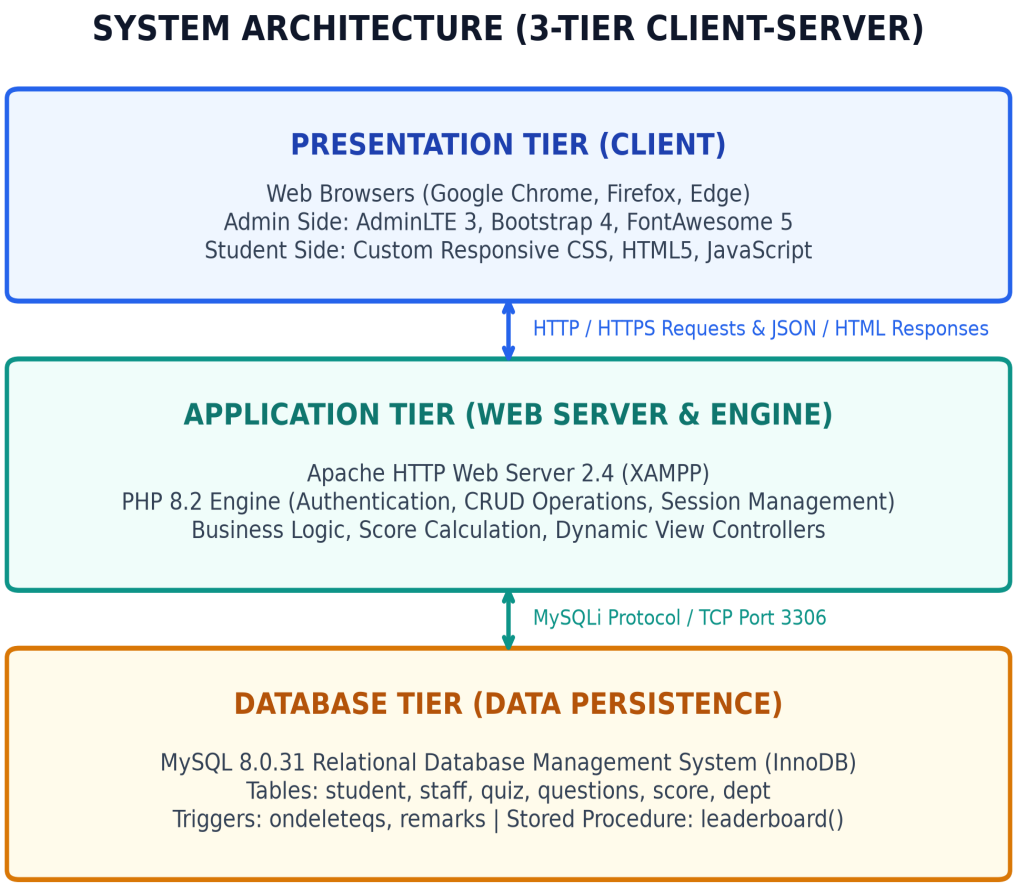


Figure 5.1: Three-Tier System Architecture of Quiz Management System

5.2 Use Case Diagram

Actors and use cases to show:

- **Staff/Admin:** Register, log in, manage profile, create quiz, add questions, view/edit/delete quiz content, view leaderboard.
- **Student:** Register, log in, view available quizzes, attempt quiz, submit answers, view scores, view leaderboard, view profile.
- **System:** Validate credentials, evaluate submitted answers, save scores, and display results.

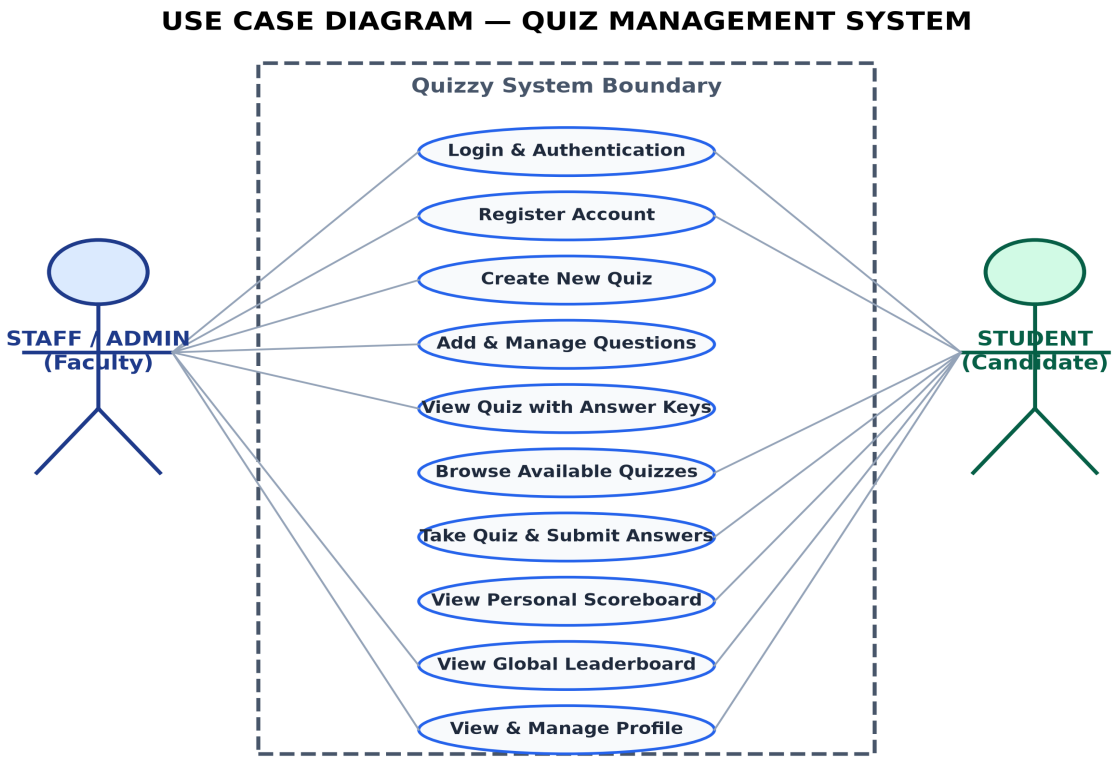


Figure 5.2: UML Use Case Diagram for Quiz Management System

5.3 Activity Diagram

The activity diagram represents the main workflow from login through quiz completion.

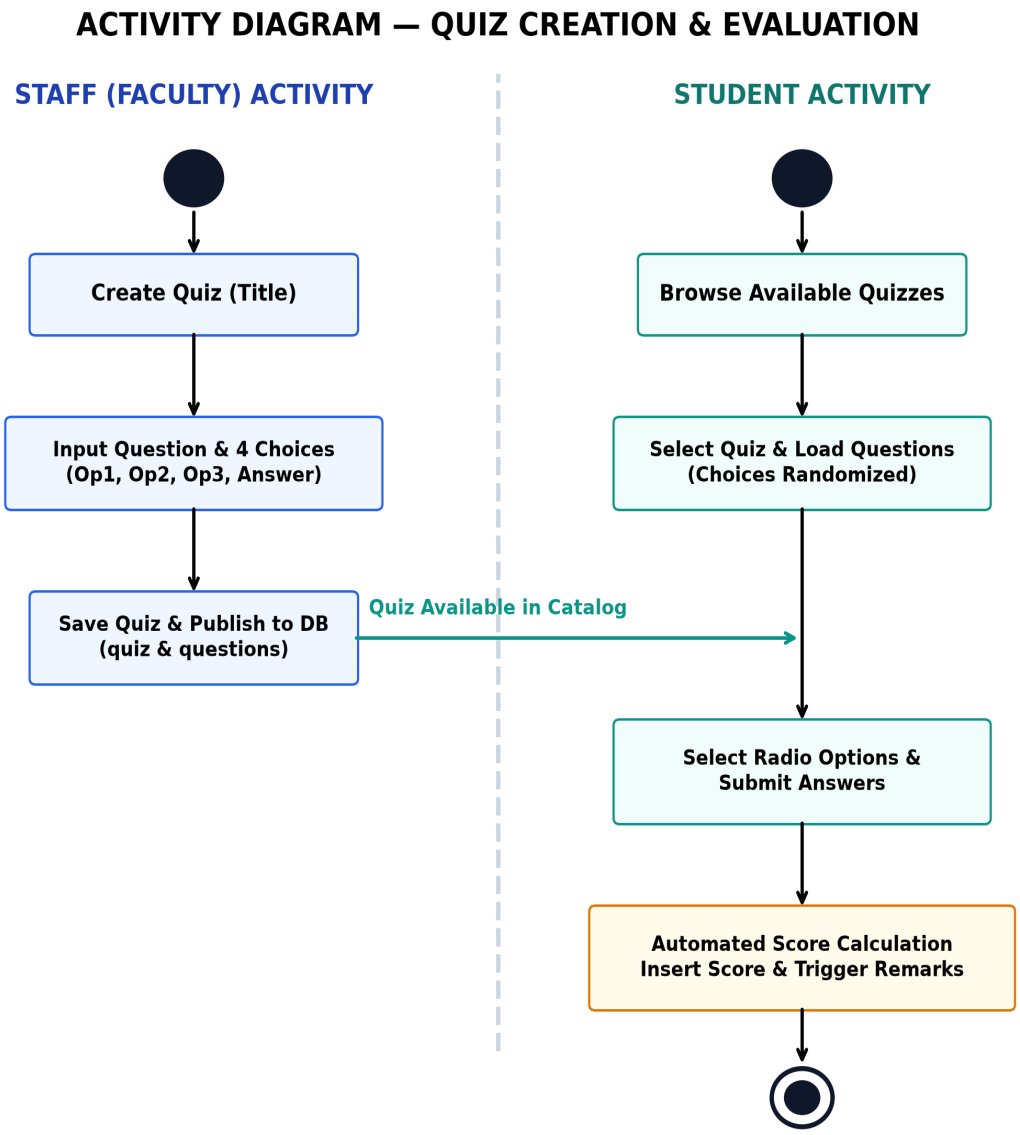


Figure 5.3: UML Activity Diagram Showing Staff and Student Workflows

5.4 ER Diagram

The ER diagram shows the main database entities and their logical relationships.

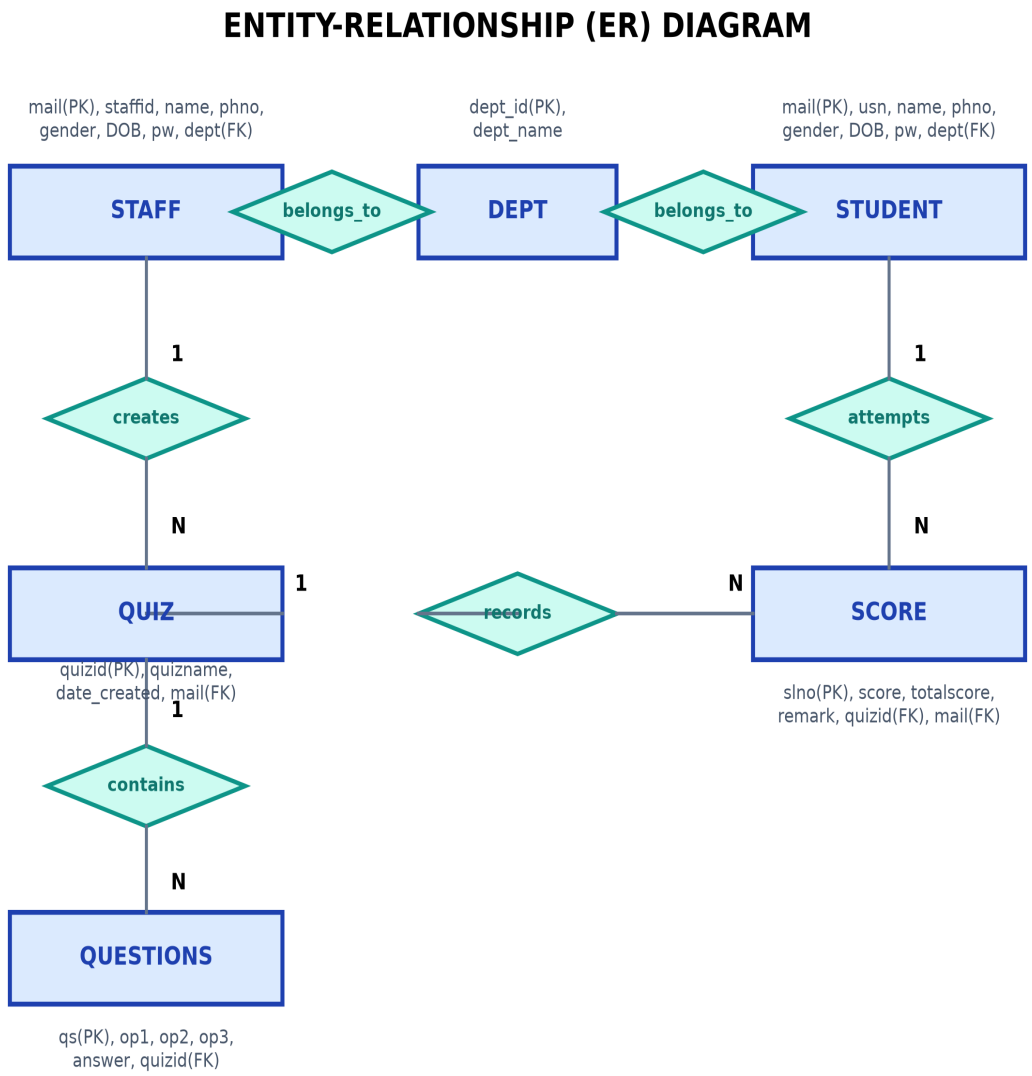


Figure 5.4: Entity Relationship (ER) Diagram of Quiz Management System Database

5.5 Relational Schema

The database uses the following relations. Underline the primary key in the report diagram or schema listing.

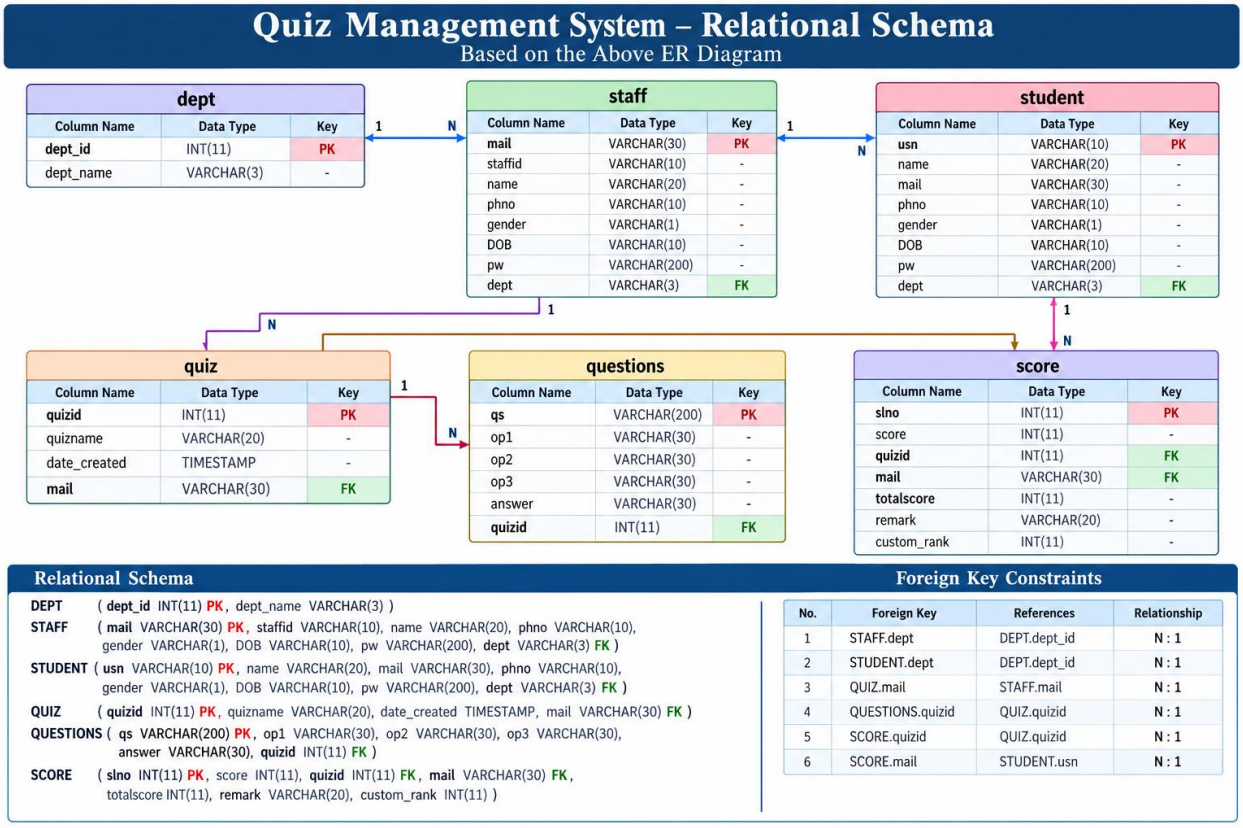


Figure 5.5: Relational Database Schema & Foreign Key Map

5.6 Context Diagram

The context diagram shows the application as one process and the data exchanged with its external users.

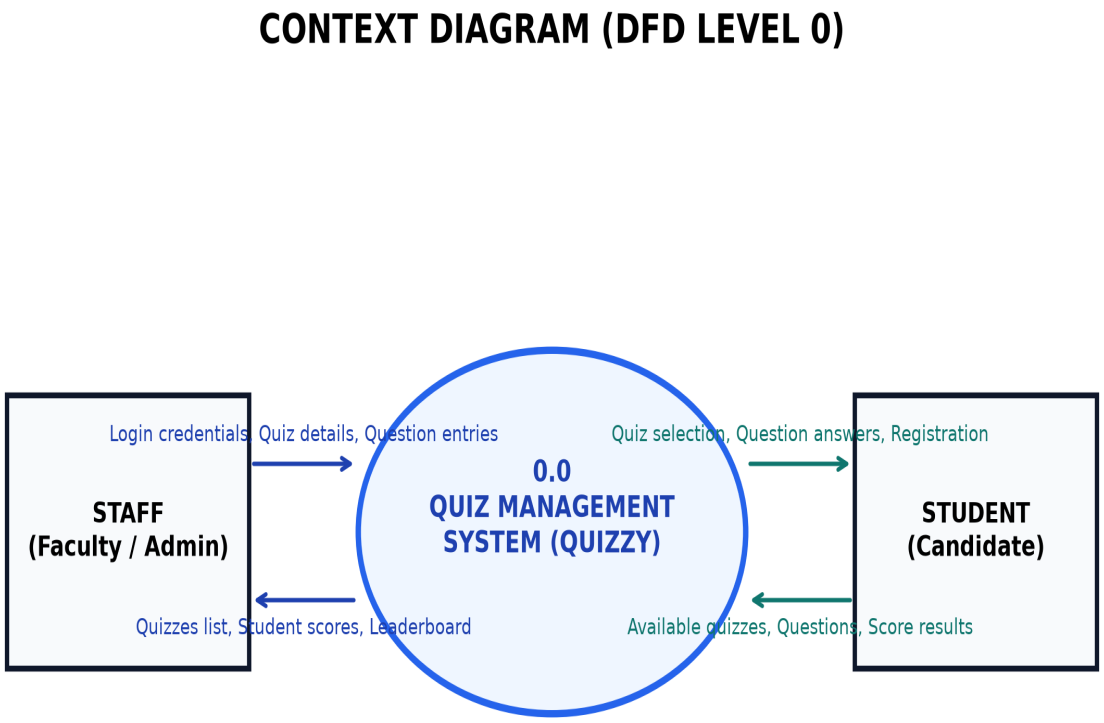


Figure 5.6: System Context Diagram (Level 0 Boundary)

5.7 System Flowchart

The system flowchart summarizes the application's major decisions and actions.

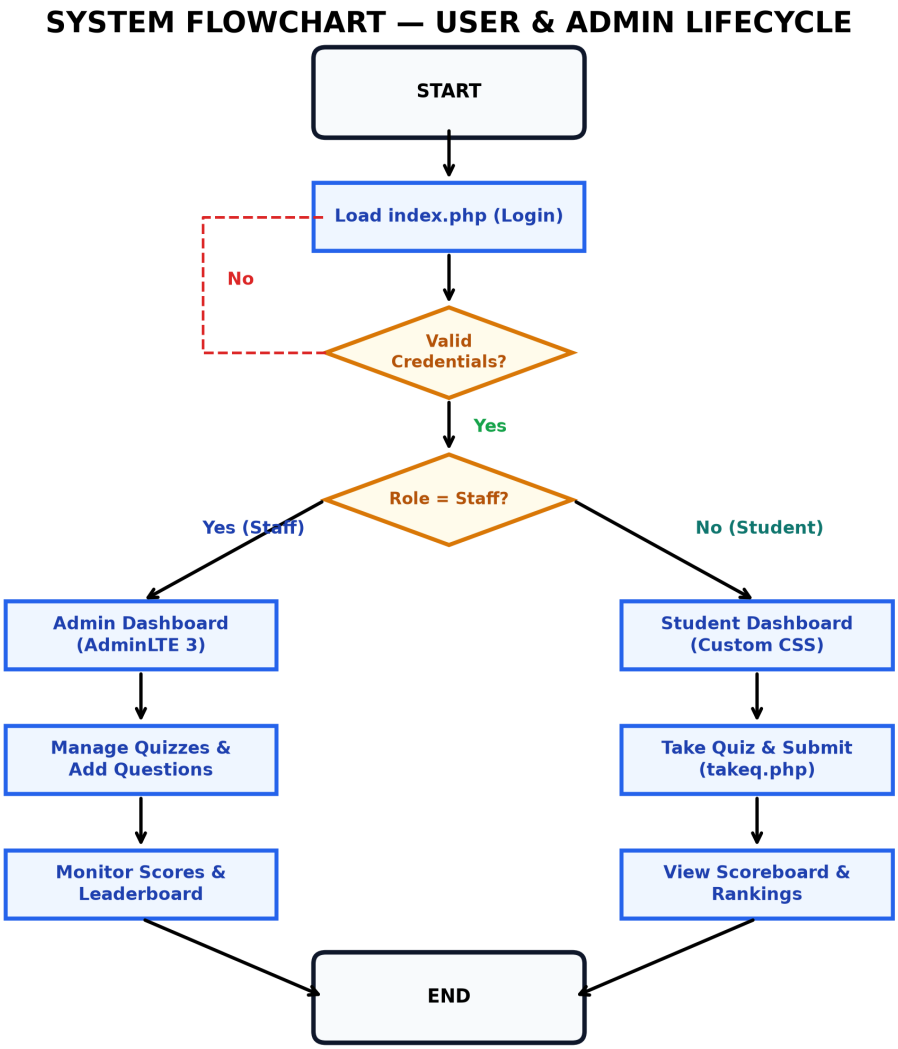


Figure 5.7: System Procedural Flowchart

5.8 Data Flow Diagram

The Data Flow Diagram (DFD) shows how information moves between users, processes, and database stores. Use conventional DFD symbols: rectangles for external entities, circles or rounded rectangles for processes, and open-ended rectangles for data stores.

5.8.1 DFD Level 0

The Level 0 DFD presents the application's main data flows at a high level.

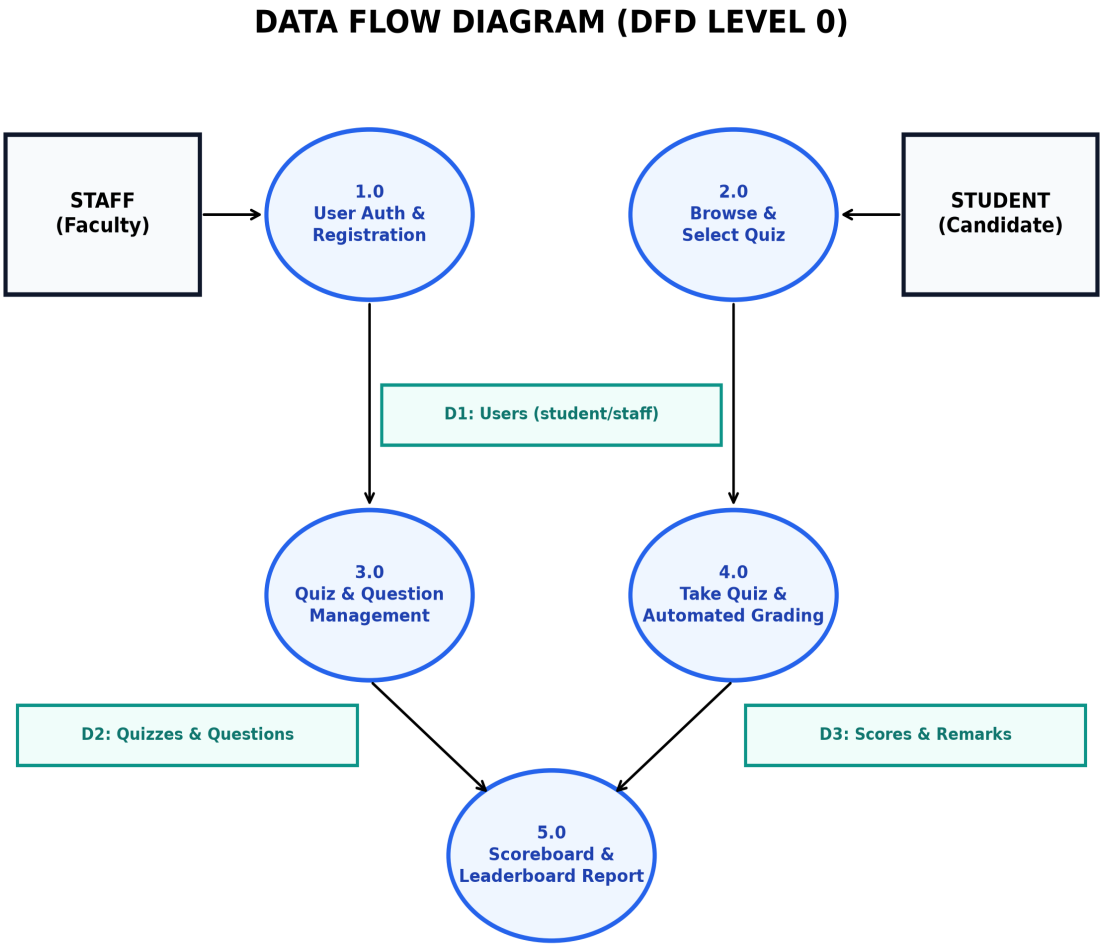


Figure 5.8: Data Flow Diagram (DFD) Level 0

5.8.2 DFD Level 1

The Level 1 DFD breaks the main system into its key processes.

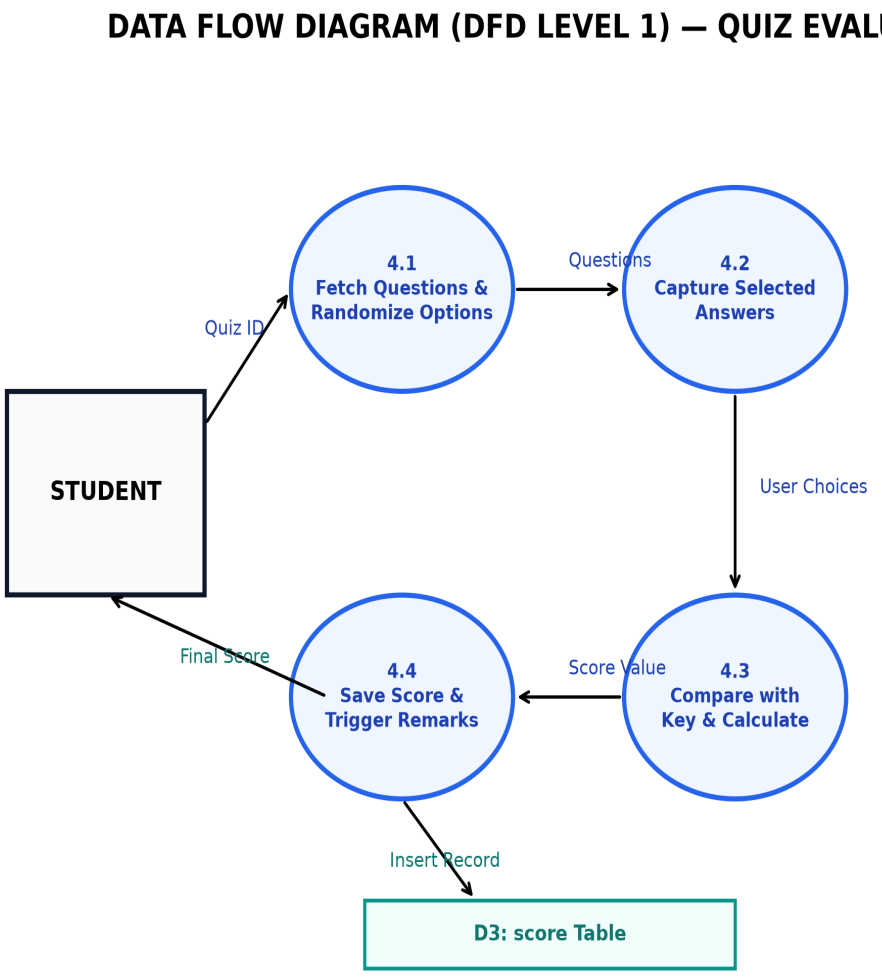


Figure 5.9: Data Flow Diagram (DFD) Level 1 Subsystem Decomposition

6.Data Dictionary

6.1 Database Overview

The database stores department, staff, student, quiz, question, and score information. The application uses these records for account access, quiz management, question display, automatic evaluation, and results.

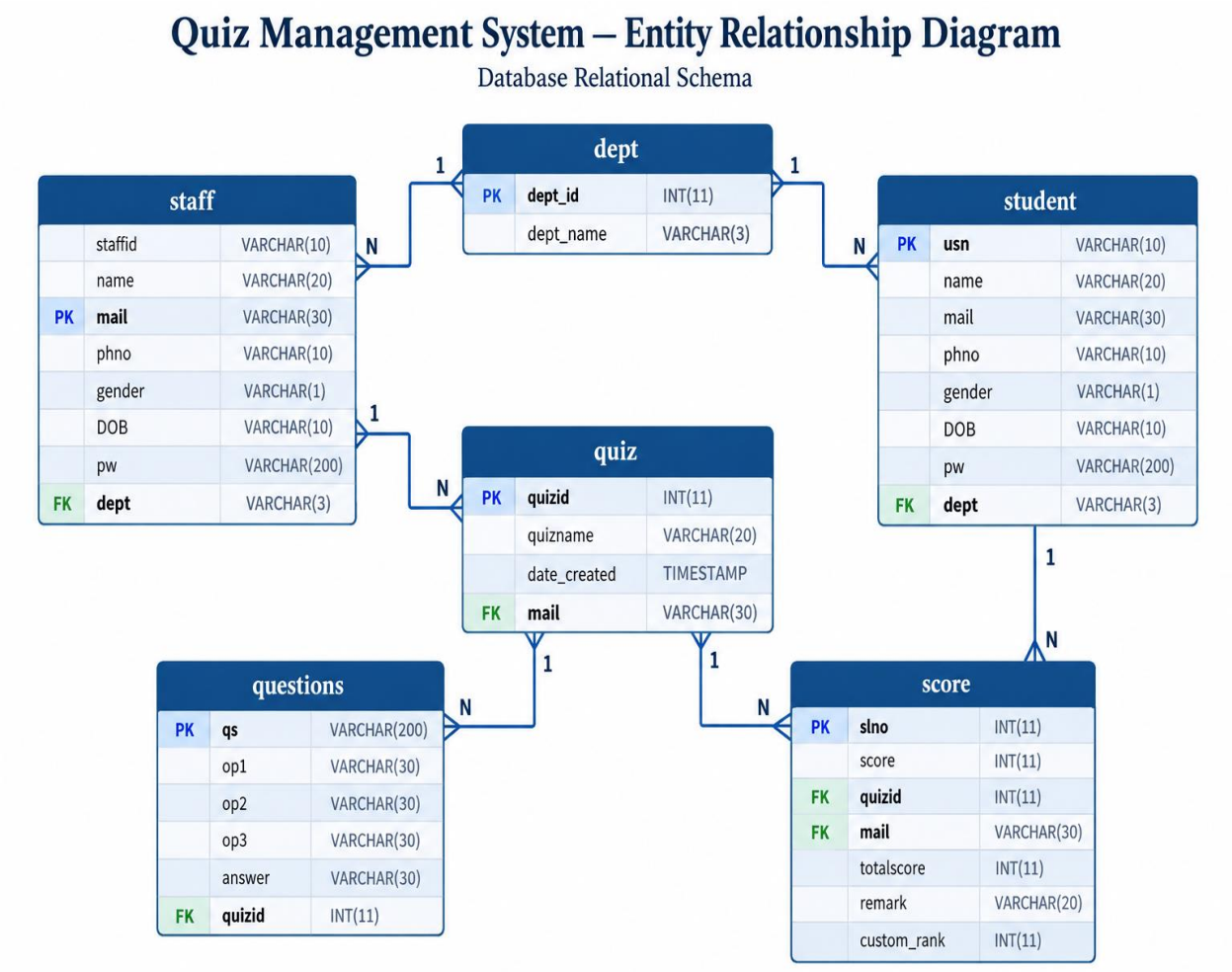


Figure 6.1: Complete Relational Database Architecture & Cardinality Map

6.2 Department Table

❖ dept

Table Name		dept		
Description		Academic Department Master Directory		
Primary Key		dept_id		
Key	**Field Name**	**Type**	**Size**	**Constraints**
*	dept_id	INT	11	PRIMARY KEY, NOT NULL
	dept_name	VARCHAR	3	DEFAULT NULL, Department Code (e.g. CSE, ISE)

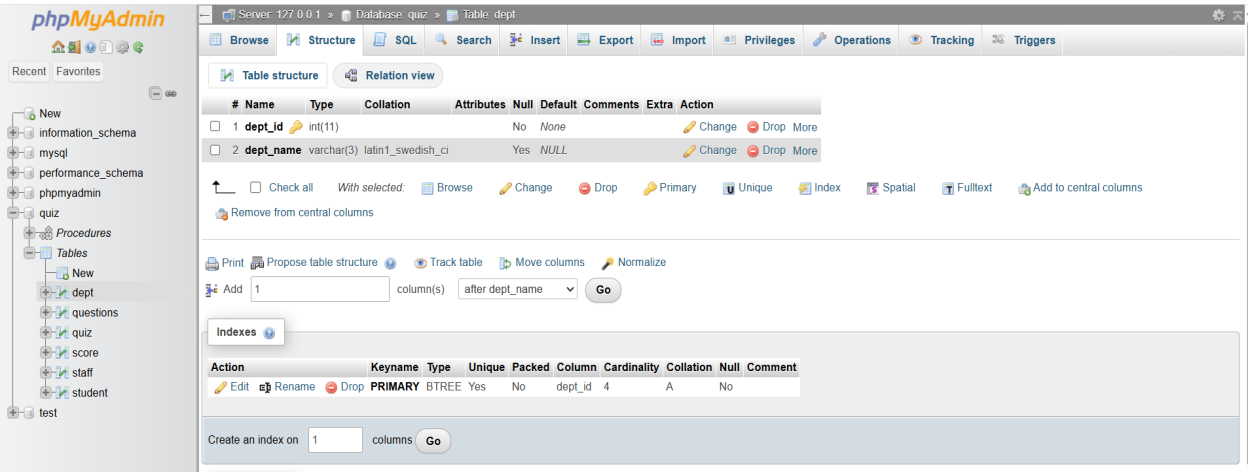


Figure 6.2: Structural Specification & Relational Map — `dept`

6.3 Staff Table

❖ staff

Table Name		staff		
Description		Faculty and Staff Administrator Records		
Primary Key		mail		
Key	**Field Name**	**Type**	**Size**	**Constraints**
*	mail	VARCHAR	30	PRIMARY KEY, Unique Official Institutional Email
UQ	staffid	VARCHAR	10	UNIQUE KEY, NOT NULL, Staff ID
	name	VARCHAR	20	NOT NULL, Full Name of Staff Member
UQ	phno	VARCHAR	10	UNIQUE KEY, NOT NULL, Contact Number
	gender	VARCHAR	1	NOT NULL, Gender Designation ('M'/'F'/'O')
	DOB	VARCHAR	10	NOT NULL, Date of Birth (YYYY-MM-DD)
	pw	VARCHAR	200	NOT NULL,

				Hashed Password Value
FK	dept	VARCHAR	3	FOREIGN KEY REFERENCES dept(dept_name)
	avatar	VARCHAR	255	DEFAULT NULL, Relative Path to Profile Avatar Photo

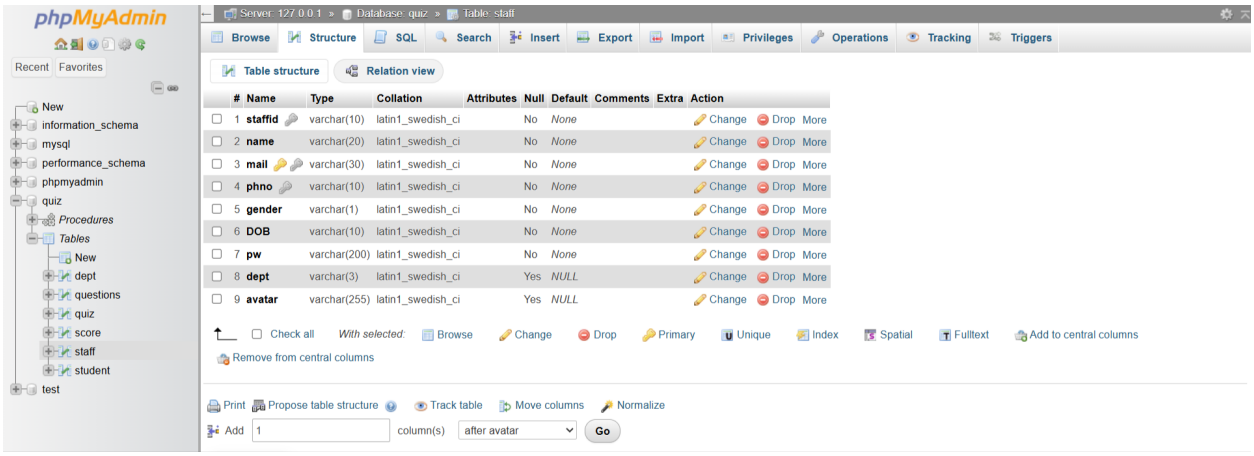


Figure 6.3: Structural Specification & Relational Map — `staff`

6.4 Student Table

❖ student

Table Name		student		
Description		Enrolled Student Examinee Records		
Primary Key		mail		
Key	**Field Name**	**Type**	**Size**	**Constraints**
*	mail	VARCHAR	30	PRIMARY KEY, Unique Student Email
UQ	usn	VARCHAR	10	UNIQUE KEY, NOT NULL, University Seat Number
	name	VARCHAR	20	NOT NULL, Full Name of Student
UQ	phno	VARCHAR	10	UNIQUE KEY, NOT NULL, Mobile Phone Number
	gender	VARCHAR	1	NOT NULL, Gender Designation ('M'/'F')
	DOB	VARCHAR	10	NOT NULL, Date of Birth (YYYY-MM-DD)
	pw	VARCHAR	200	NOT NULL, Cryptographically

				Salted Password Hash
FK	dept	VARCHAR	3	FOREIGN KEY REFERENCES dept(dept_name)

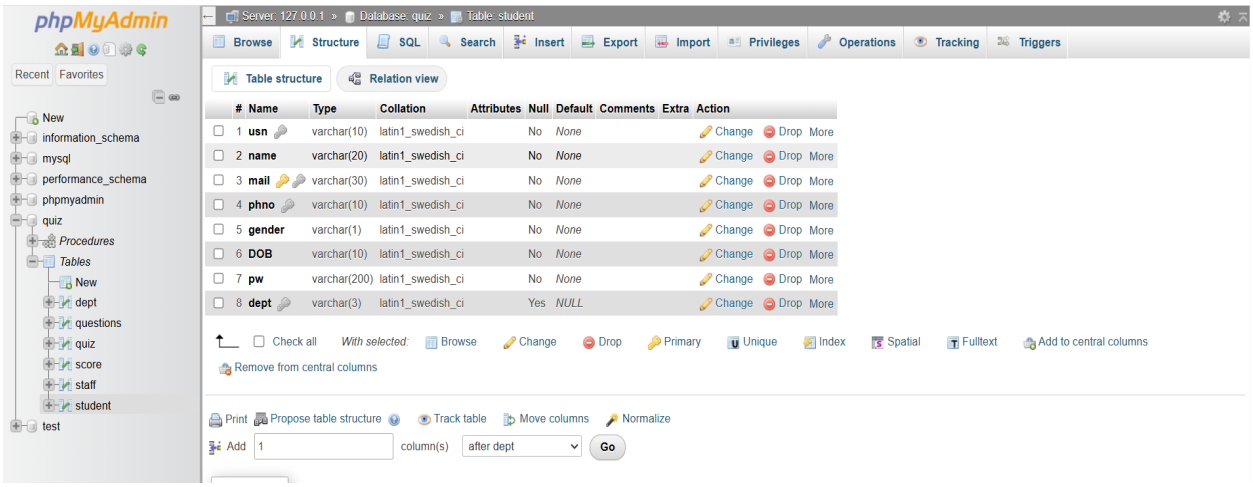


Figure 6.4: Structural Specification & Relational Map — `student`

6.5 Quiz Table

❖ quiz

Table Name		quiz		
Description		Quiz Header and Metadata Records		
Primary Key		quizid		
Key	**Field Name**	**Type**	**Size**	**Constraints**
*	quizid	INT	11	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
	quizname	VARCHAR	20	NOT NULL, Human-Readable Quiz Title
	date_created	TIMESTAMP	—	NOT NULL, DEFAULT CURRENT_TIMESTAMP
FK	mail	VARCHAR	30	FOREIGN KEY REFERENCES staff(mail) ON DELETE CASCADE

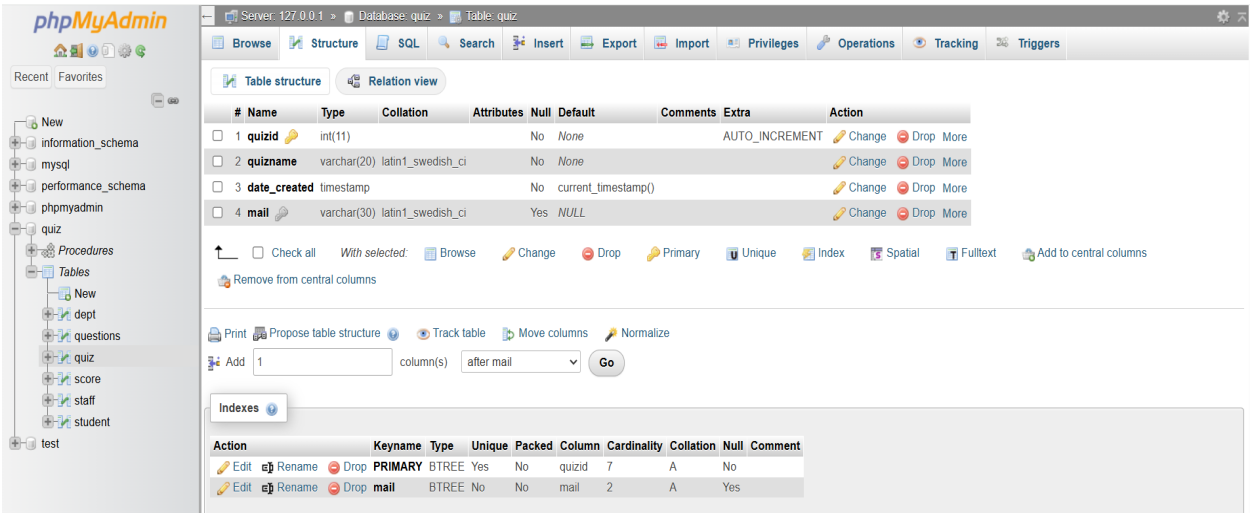


Figure 6.5: Structural Specification & Relational Map — `quiz`

6.6 Questions Table

❖ questions

Table Name		questions		
Description		Question Bank with Options and Correct Answers		
Primary Key		qs		
Key	**Field Name**	**Type**	**Size**	**Constraints**
*	qs	VARCHAR	200	PRIMARY KEY, UNIQUE KEY, Question Text
	op1	VARCHAR	30	NOT NULL, Distractor Option 1
	op2	VARCHAR	30	NOT NULL, Distractor Option 2
	op3	VARCHAR	30	NOT NULL, Distractor Option 3
	answer	VARCHAR	30	NOT NULL, Authoritative Correct Answer

FK	quizid	INT	11	FOREIGN KEY REFERENCES quiz(quizid) ON DELETE CASCADE

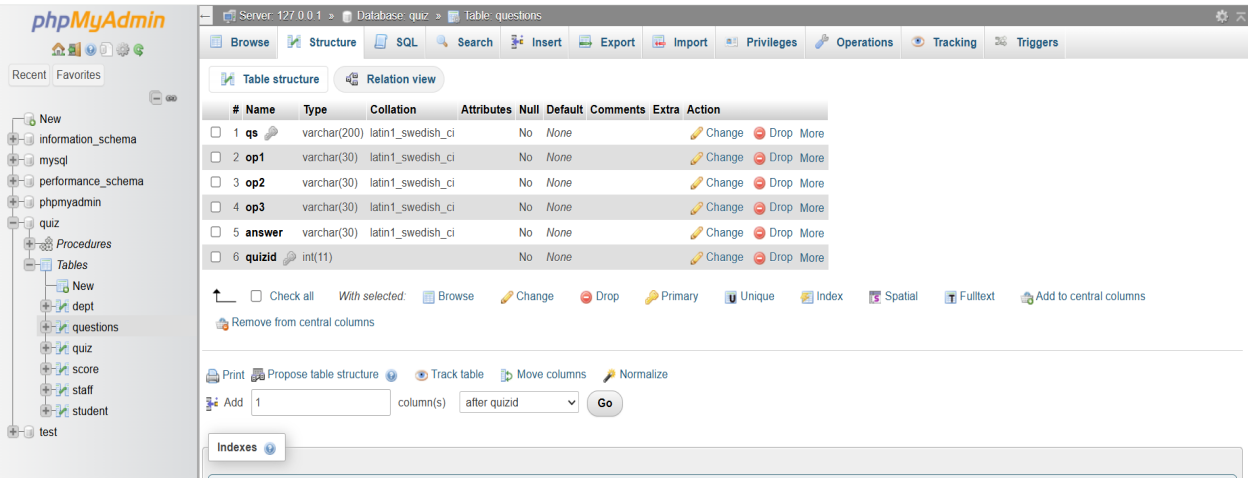


Figure 6.6: Structural Specification & Relational Map — `questions`

6.7 Score Table

❖ score

Table Name		score		
Description		Student Examination Results and Automated Remarks		
Primary Key		slno		
Key	**Field Name**	**Type**	**Size**	**Constraints**
*	slno	INT	11	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
	score	INT	11	NOT NULL, Absolute Marks Obtained by Student
FK	quizid	INT	11	FOREIGN KEY REFERENCES quiz(quizid) ON DELETE CASCADE
FK	mail	VARCHAR	30	FOREIGN KEY REFERENCES student(mail) ON DELETE CASCADE
	totalscore	INT	11	DEFAULT NULL, Maximum Possible Marks
	remark	VARCHAR	20	Assigned via Database Trigger `remarks` ('good'/'bad')
	custom_rank	INT	11	DEFAULT NULL, Administrator-

				Assigned Custom Display Rank

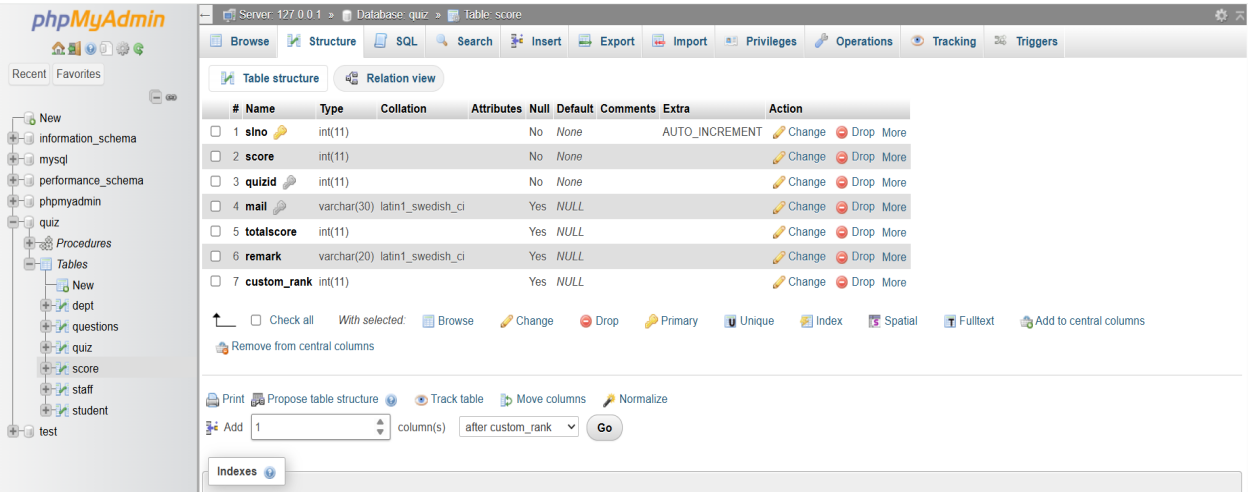


Figure 6.7: Structural Specification & Relational Map — `score`

6.8 Stored Procedures and Triggers

The database includes the leaderboard stored procedure. It joins score, student, and quiz data, then orders the results by score in descending order. The ondeleteqs trigger runs after a quiz is deleted and removes its associated questions. The remarks trigger runs before a score is inserted: it sets the remark to bad when the score is zero and good otherwise.

6.9 Referential Integrity and Normalization

The schema defines foreign keys from quiz.mail to staff.mail, from score.quizid to quiz.quizid, and from score.mail to student.mail. Deleting a referenced staff member, quiz, or student can cascade to related quiz or score records according to the configured constraints. The quiz-deletion trigger also removes related question records.

7.Implementation Screenshots

7.1 Staff / Admin Interface

7.1.1 Staff / Admin Login Interface

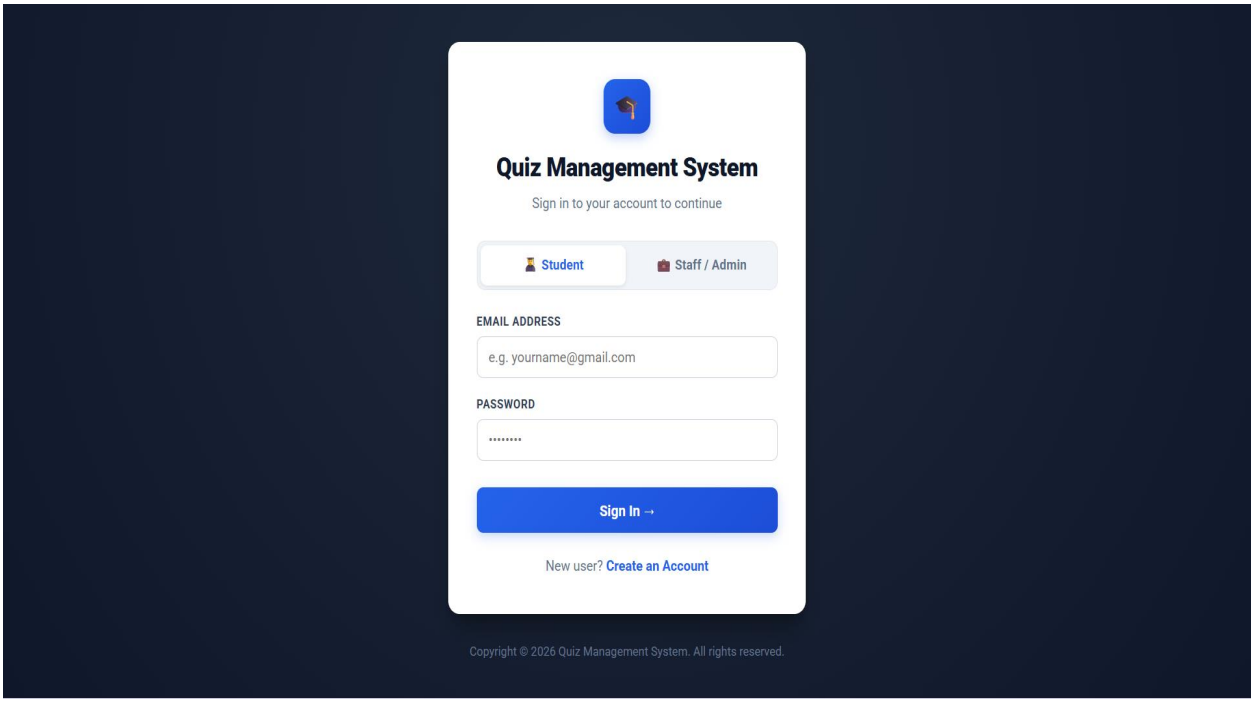


Figure 7.1.1: Login page of Quiz Management System

Source: index.php

```
<?php
if ($type === 'student' || $type === 'staff') {
    $sql = "SELECT * FROM " . $type . " WHERE
mail='{$username}'";
    $res = mysqli_query($conn, $sql);
    if ($res && mysqli_num_rows($res) > 0) {
```

```
        $row = mysqli_fetch_array($res);
        if ($row['pw'] === $password) {
            $_SESSION['type'] = $type;
            $_SESSION['username'] = $row['mail'];
            header("Location: " . ($type === 'student'
                ? 'homestud.php' : 'homestaff.php'));
            exit();
        }
    }
}
```

7.1.2 Staff / Administrator Registration Page

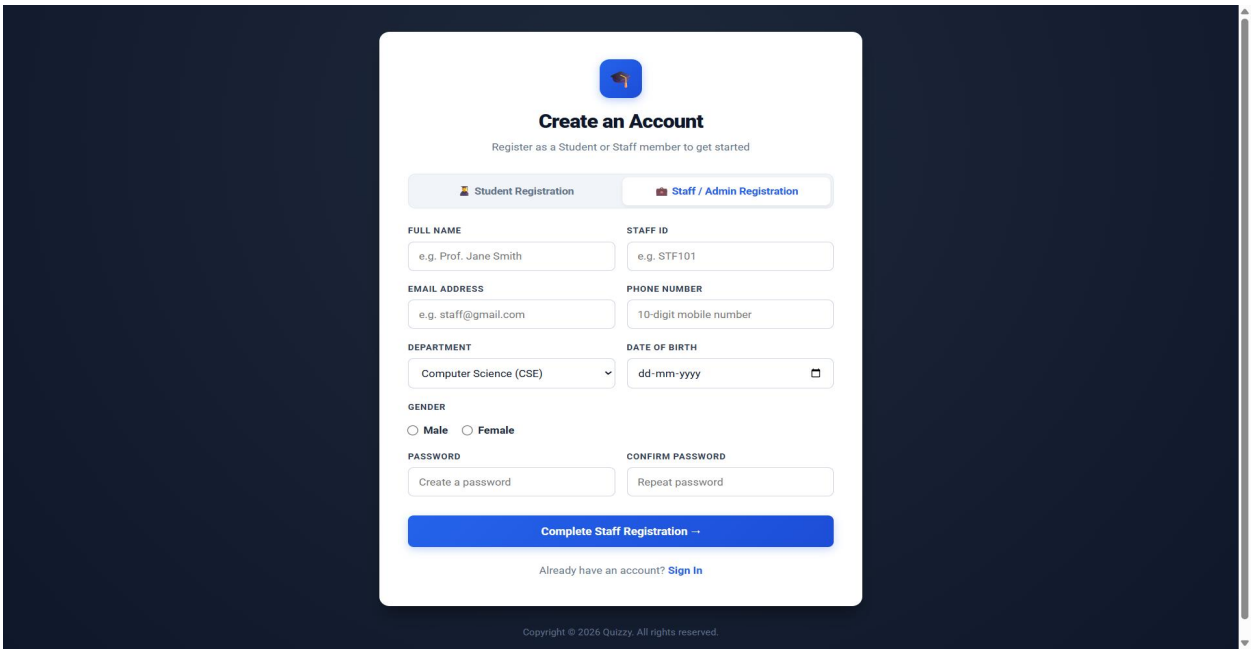


Figure 7.1.2: Staff / Admin registration page

Source: signup.php

```
<?php
if ($password2 === $cpassword2) {
    $password2_hash = crypt($password2,
'rakeshmariyaplarrakesh');


    $sql = "INSERT INTO staff
        (staffid, name, mail, phno, dept, gender, DOB, pw)
        VALUES ('$staffid', '$name2', '$mail2', '$phno2',
                '$dept2', '$gender2', '$dob2',
'$password2_hash')";
    mysqli_query($conn, $sql);
}
```

7.1.3 Staff / Admin Dashboard

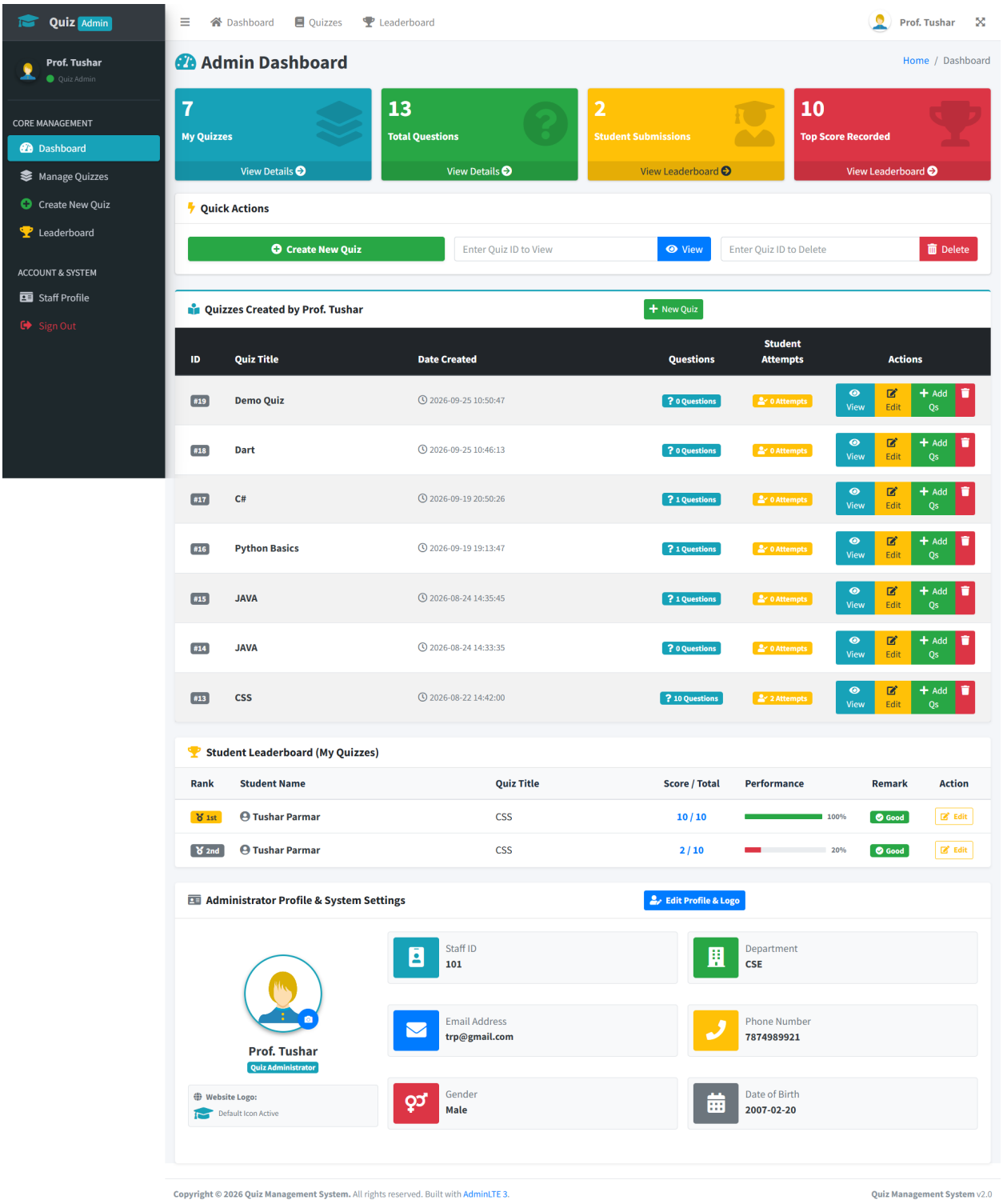


Figure 7.1.3: Staff / Admin dashboard

Source: homestaff.php

```
<?php
$res = mysqli_query(
    $conn,
    "SELECT COUNT(*) AS cnt FROM quiz WHERE
mail = '$username1'"
);
$stat_quizzes = mysqli_fetch_assoc($res)['cnt'] ?? 0;


$res = mysqli_query(
    $conn,
    "SELECT MAX(sc.score) AS mscore
     FROM score sc JOIN quiz q ON sc.quizid = q.quizid
     WHERE q.mail = '$username1'"
);
$stat_max_score = mysqli_fetch_assoc($res)['mscore'] ??
0;
```

7.1.4 Quiz Management Table

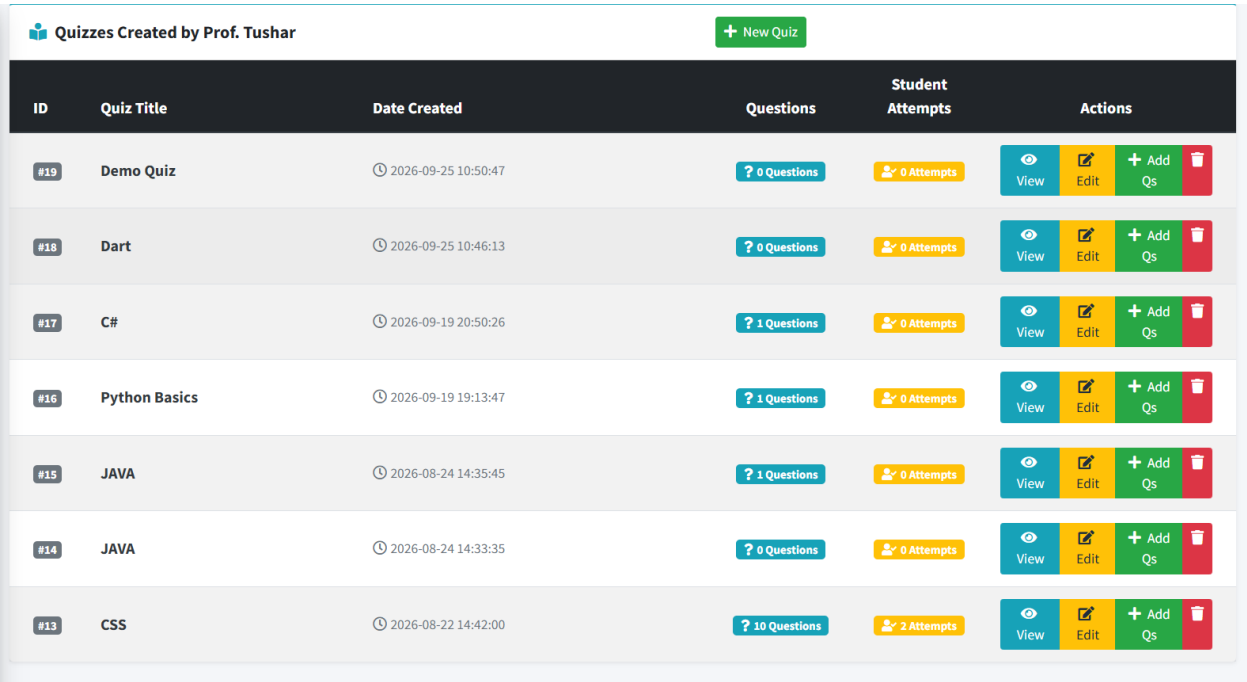


Figure 7.1.4: Quiz management table in the staff/admin dashboard

Source: homestaff.php

```
<?php

$quizzes_query = "SELECT q.quizid, q.quizname,
q.date_created,

    (SELECT COUNT(*) FROM questions qs WHERE
qs.quizid = q.quizid) AS q_count,

    (SELECT COUNT(*) FROM score sc WHERE
sc.quizid = q.quizid) AS a_count

    FROM quiz q WHERE q.mail = '$username1'
```

```
    ORDER BY q.quizid DESC";
$quizzes_res = mysqli_query($conn, $quizzes_query);
```

7.1.5 Create New Quiz

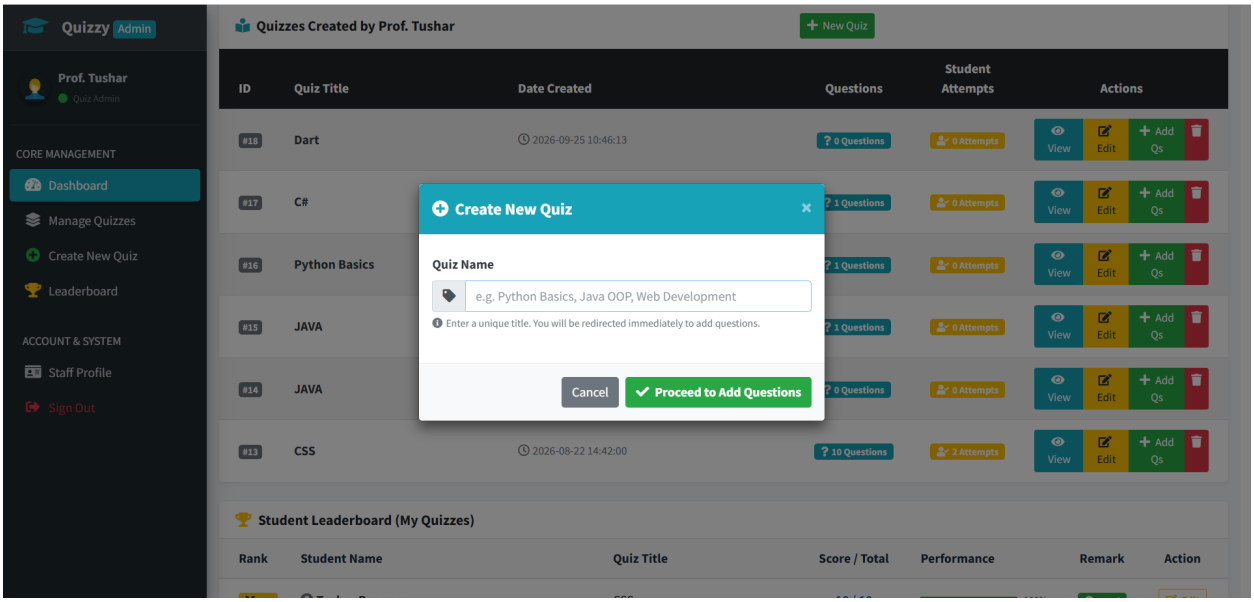


Figure 7.1.5: Create New Quiz dialog

Source: homestaff.php

```
<?php
if (isset($_POST['submit'])) {
    $qname = strtolower(trim($_POST['quizname']));
    $qname_clean = mysqli_real_escape_string($conn, $qname);
    $sql = "INSERT INTO quiz (quizname, mail)
```

```
        VALUES ('$qname_clean', '$username1')";

    if (mysqli_query($conn, $sql)) {
        $_SESSION['qname'] = $qname;
        header('Location: addqs.php');
        exit();
    }
}
```

7.1.6 Add Questions to Quiz

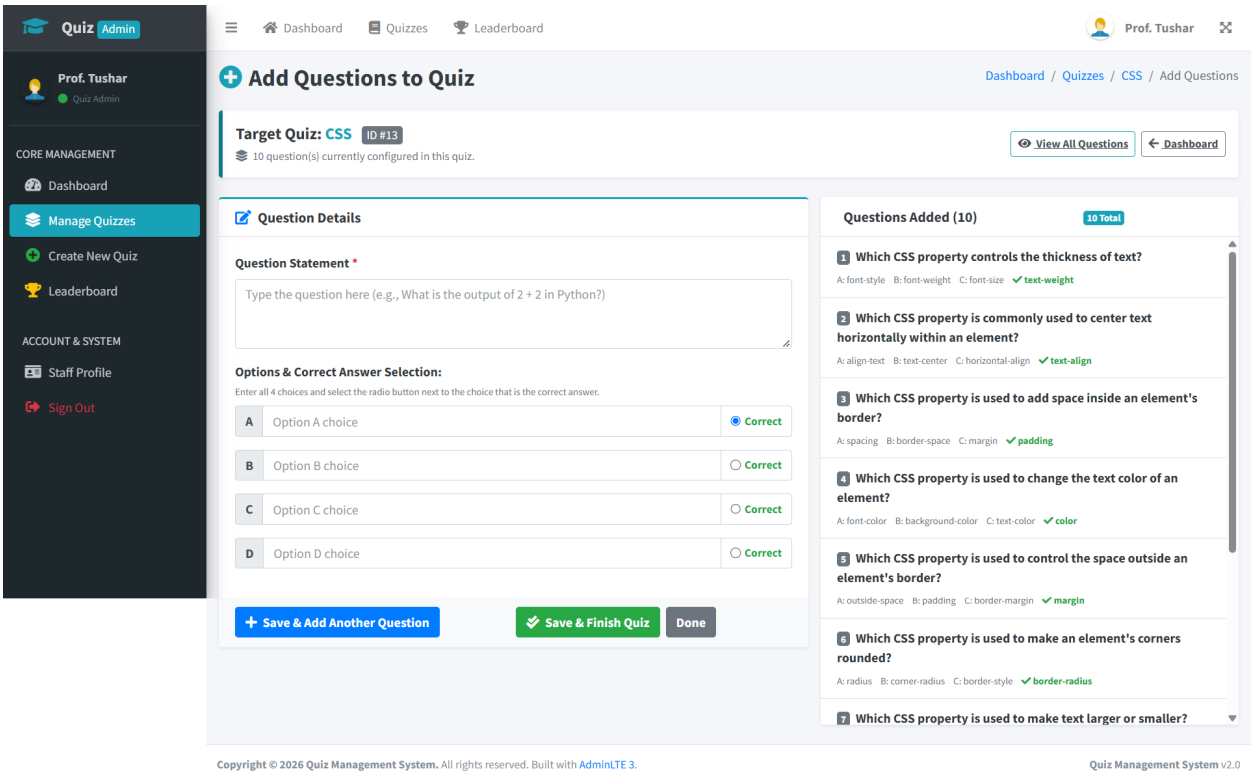


Figure 7.1.6: Add questions to quiz

Source: addqs.php

```
<?php
$correct_choice = $_POST['correct_opt'] ?? 'A';


if ($correct_choice === 'A') {
    $ans = $opt_a;
    $options = [$opt_b, $opt_c, $opt_d];
} elseif ($correct_choice === 'B') {
    $ans = $opt_b;
    $options = [$opt_a, $opt_c, $opt_d];
} elseif ($correct_choice === 'C') {
    $ans = $opt_c;
    $options = [$opt_a, $opt_b, $opt_d];
} else {
    $ans = $opt_d;
    $options = [$opt_a, $opt_b, $opt_c];
}
```

```
$insert_sql = "INSERT INTO questions (qs, op1, op2, op3, answer, quizid)
                VALUES ('$qs_c', '$op1_c', '$op2_c', '$op3_c', '$ans_c', '$qid')";
mysqli_query($conn, $insert_sql);
```

7.1.7: Quiz Questions

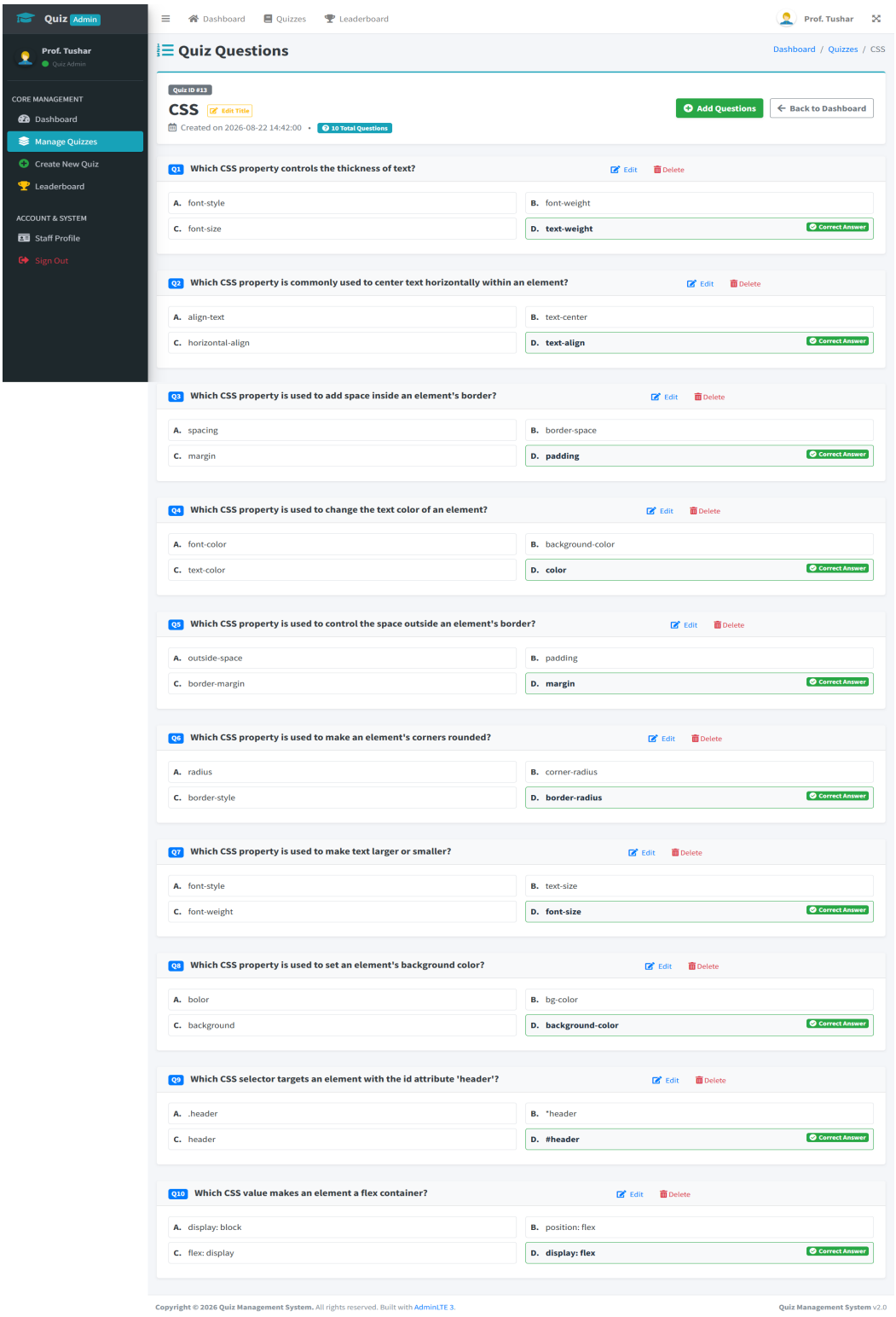


Figure 7.1.7: Quiz questions

Source: viewq.php

```
<?php

$qs_sql = "SELECT * FROM questions WHERE quizid
= '$qid'";

$qs_res = mysqli_query($conn, $qs_sql);

$total_questions = mysqli_num_rows($qs_res);


while ($row = mysqli_fetch_assoc($qs_res)) {
    echo htmlspecialchars($row['qs']);
    echo htmlspecialchars($row['op1']);
    echo htmlspecialchars($row['op2']);
    echo htmlspecialchars($row['op3']);
    echo htmlspecialchars($row['answer']);
}
```

7.1.8 Student Leaderboard (Staff View)

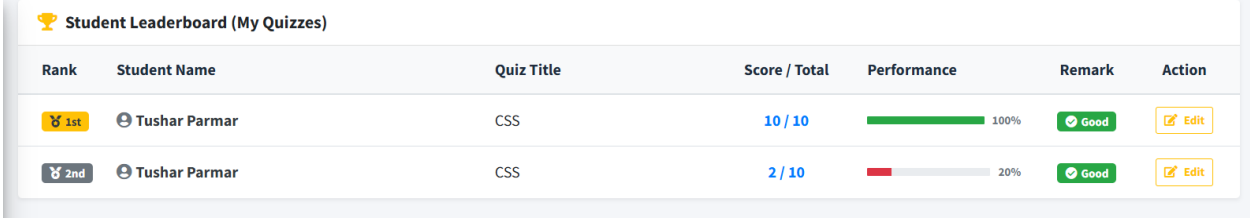


Figure 7.1.8: Student Leaderboard

Source: homestaff.php

```
<?php
$lead_query = "SELECT sc.slno, s.name, sc.score, sc.totalscore,
                    sc.remark, sc.custom_rank, q.quizname
              FROM student s, staff st, score sc, quiz q
              WHERE q.quizid = sc.quizid AND s.mail = sc.mail
                AND q.mail = st.mail AND q.mail = '$username1'
              ORDER BY sc.score DESC LIMIT 50";

$percent = $total > 0 ? round(($score / $total) * 100) : 0;
```

7.1.9 Admin Profile

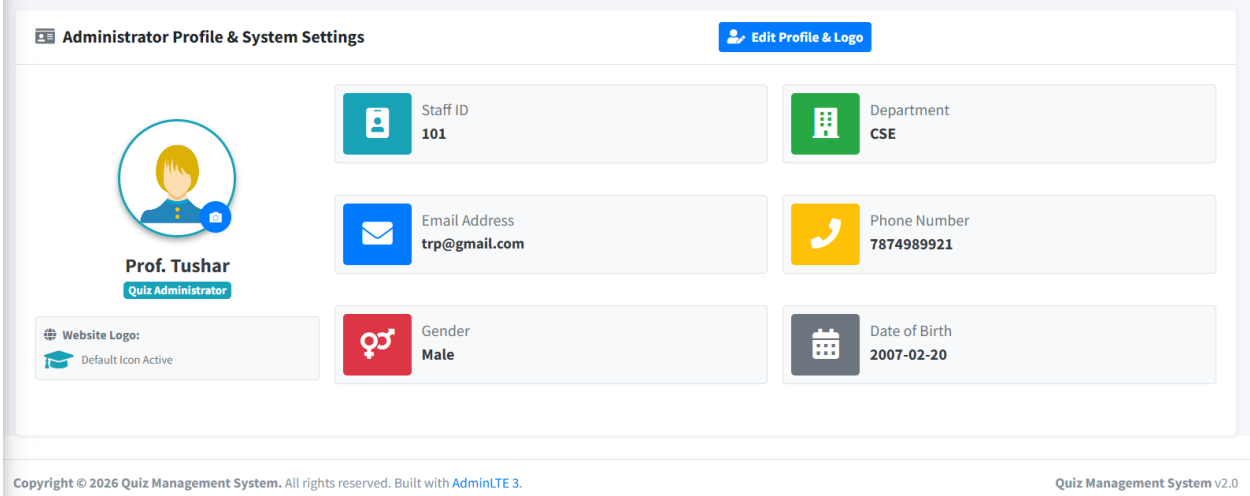


Figure 7.1.9: Admin Profile

Source: homestaff.php

```
<?php
<img src="<?= $staff_avatar ?>" alt="Staff Avatar">
<span>Staff ID: <?= $dbusn ?></span>
<span>Department: <?= $dbdept ?></span>
<span>Email Address: <?= $dbmail ?></span>
<span>Phone Number: <?= $dbphno ?></span>
```

7.1.10 Edit Administrator Profile and Website Logo

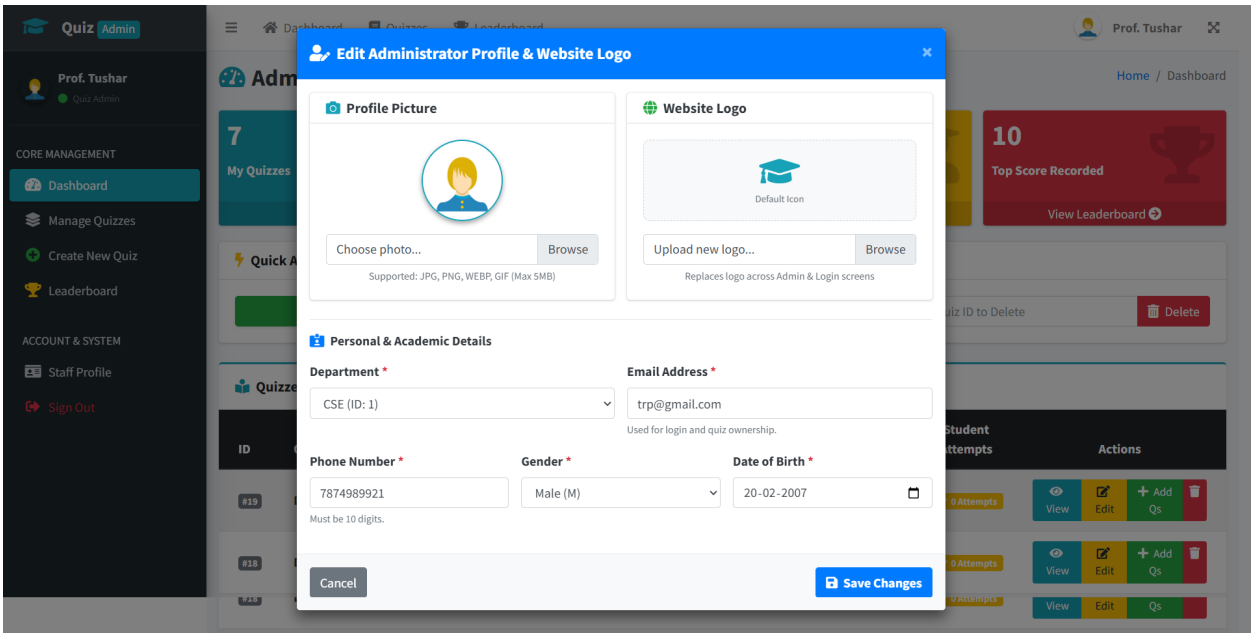


Figure 7.1.10 Edit admin profile & website logo

Source: homestaff.php

```
<form method="POST" enctype="multipart/form-data">

    <input type="file" name="profile_pic"
accept="image/*">

    <input type="file" name="site_logo"
accept="image/*">

    <select name="dept" required>

        <!-- Department options -->

    </select>
```

```
    <input type="email" name="mail" required>
    <input type="tel" name="phno" required>
    <select name="gender" required>
        <!-- Gender options -->
    </select>
    <input type="date" name="dob" required>
    <button type="submit"
name="update_profile_submit">
        Save Changes
    </button>
</form>
```

7.2 Client / Student Interface

7.2.1: Student Registration Form

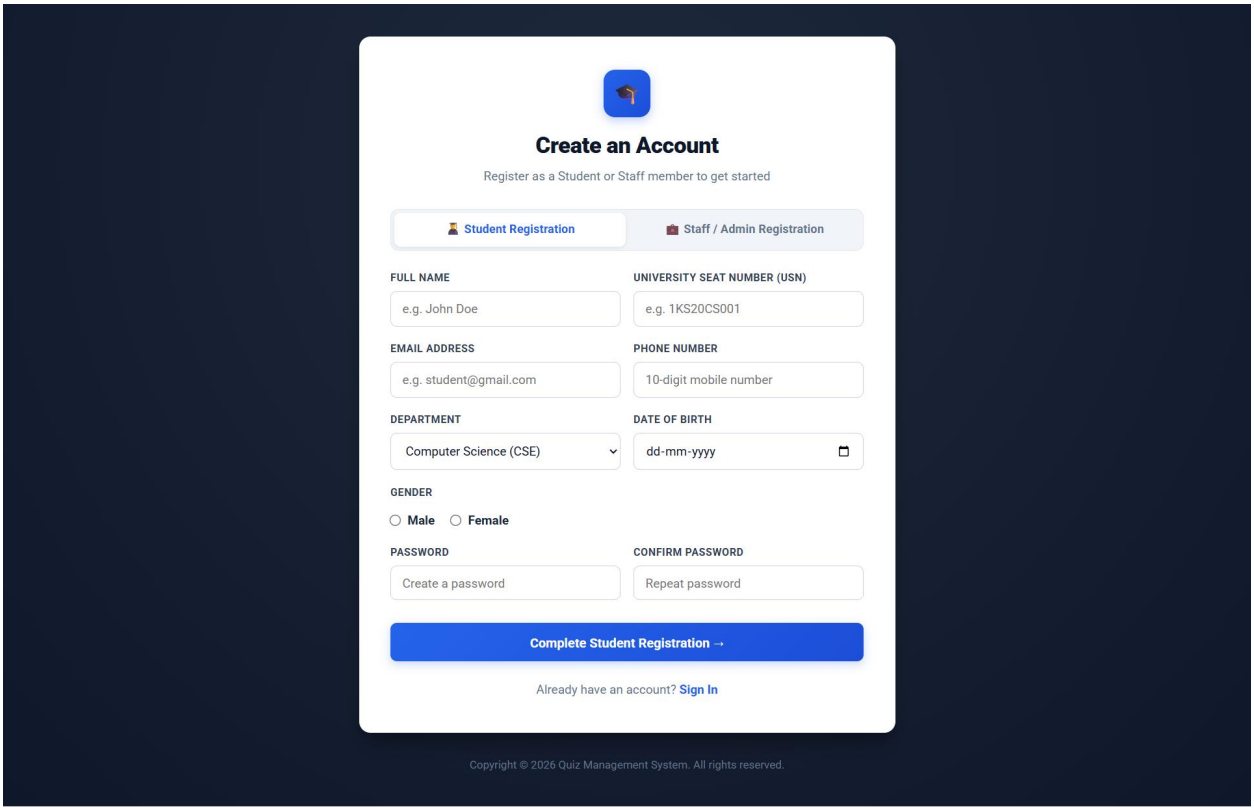


Figure 7.2.1: Student registration form

Source: signup.php

```
<?php
if ($password1 === $cpassword1) {
    $password1_hash = crypt($password1,
'rakeshmariyaplarrakesh');
    $sql = "INSERT INTO student
        (usn, name, mail, phno, dept, gender, DOB, pw)
```

```
        VALUES ('$usn1', '$name1', '$mail1', '$phno1',
                '$dept1', '$gender1', '$dob1',
'$password1_hash')";
    mysqli_query($conn, $sql);
}
```

7.2.2: Student Dashboard

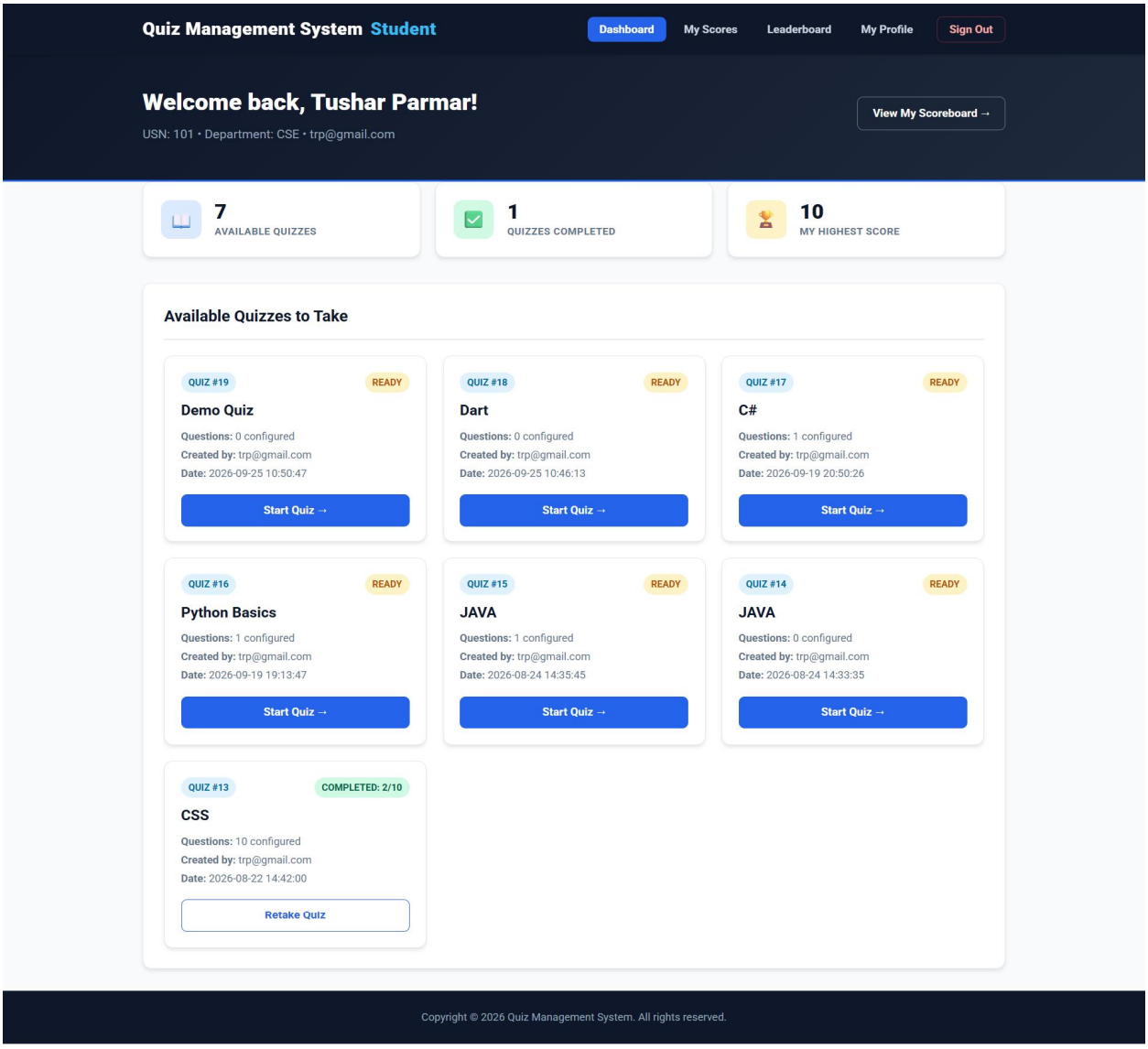


Figure 7.2.2: Student dashboard

Source: homestud.php

```
<?php
$quizzes_sql = "SELECT q.quizid, q.quizname,
q.date_created, q.mail,
    (SELECT COUNT(*) FROM questions qs WHERE
qs.quizid = q.quizid) AS q_count,
    (SELECT score FROM score sc WHERE sc.quizid =
q.quizid
      AND sc.mail = '$username1' ORDER BY sc.slno
DESC LIMIT 1) AS my_score
    FROM quiz q ORDER BY q.quizid DESC";
$quizzes_res = mysqli_query($conn, $quizzes_sql);
```

7.2.3: Student Scoreboard

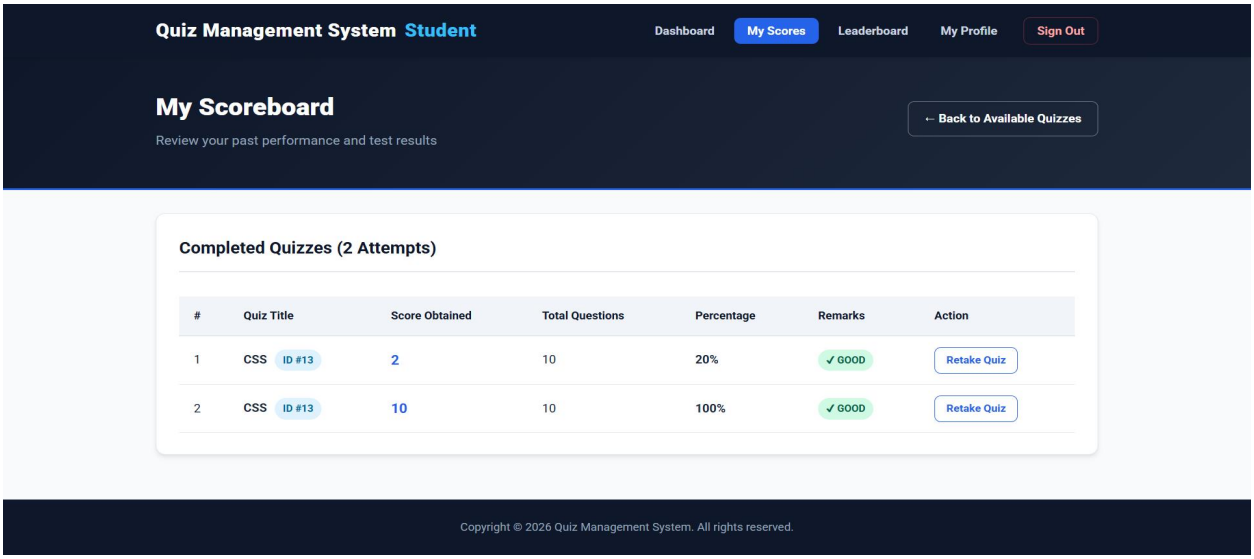


Figure 7.2.3: Student Scoreboard

Source: student_score.php

```
<?php
$score_sql = "SELECT s.score, s.totalscore, s.remark, q.quizid, q.quizname
        FROM score s JOIN quiz q ON s.quizid = q.quizid
        WHERE s.mail = '$username1'
        ORDER BY s.slno DESC";

$percent = $total > 0 ? round(($score / $total) * 100) : 0;
```

7.2.4: Student Leaderboard

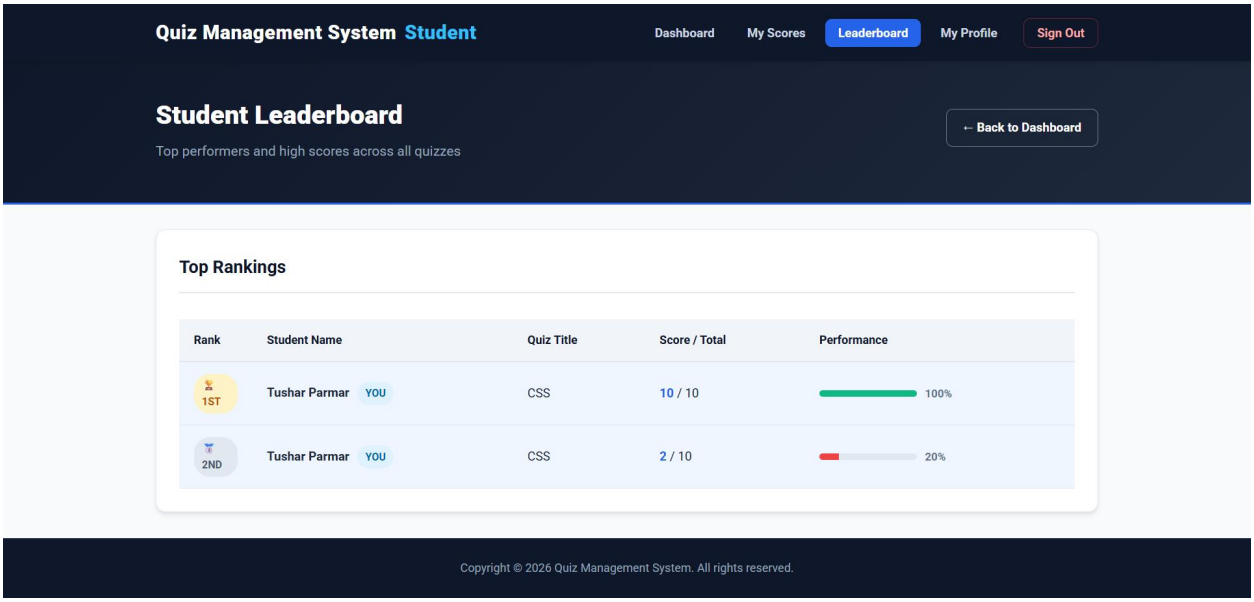


Figure 7.2.4: Student Leaderboard

Source: student_leaderboard.php

```
<?php
$lead_sql = "CALL leaderboard();";
$lead_res = mysqli_query($conn, $lead_sql);


$percent = $total > 0 ? round(($score / $total) * 100) : 0;
$is_me = ($row['mail'] === $username1);
```

7.2.5: Student Profile

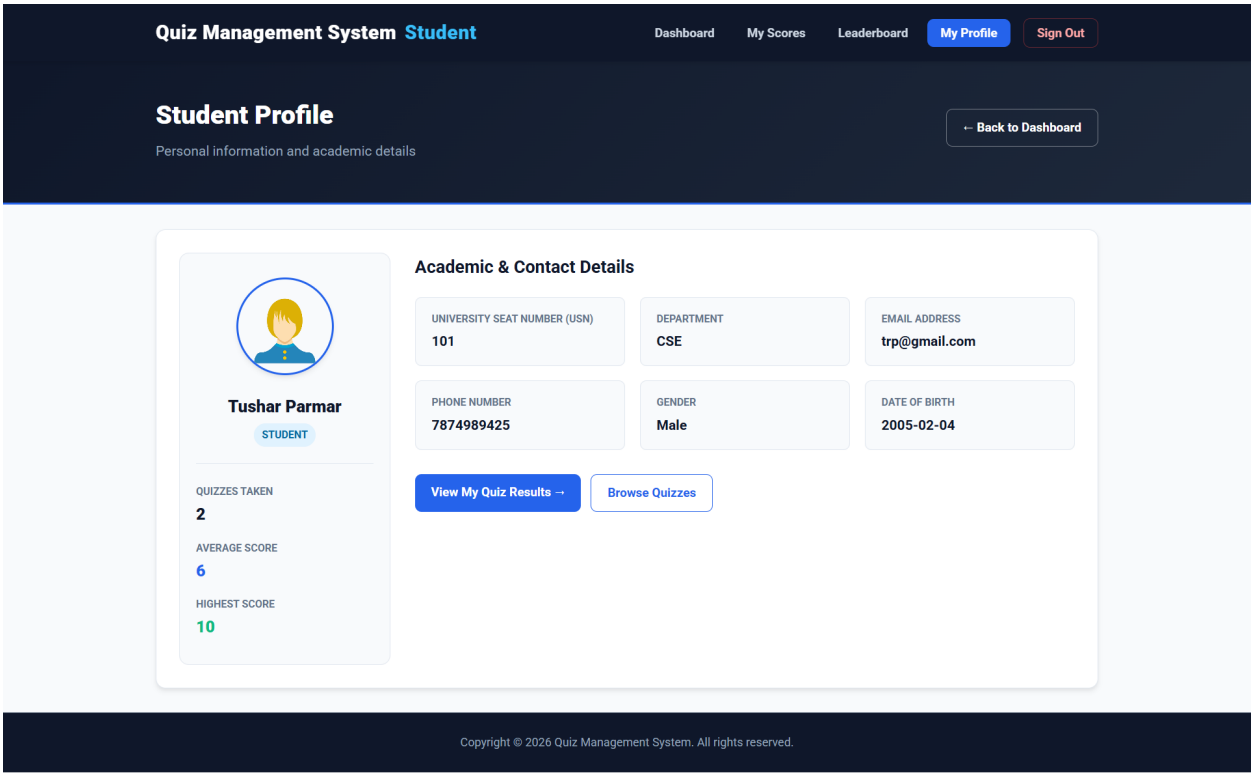


Figure 7.2.5: Student profile page

Source: student_profile.php

```
<?php
$res = mysqli_query(
    $conn,
    "SELECT COUNT(*) AS cnt FROM score WHERE mail = '$username1'"
);
$stat_completed = mysqli_fetch_assoc($res)['cnt'] ?? 0;


$res = mysqli_query(
    $conn,
    "SELECT MAX(score) AS mscore FROM score WHERE mail = '$username1'"
);
$stat_best = mysqli_fetch_assoc($res)['mscore'] ?? 0;
```

7.2.6: Student Quiz Attempt Page

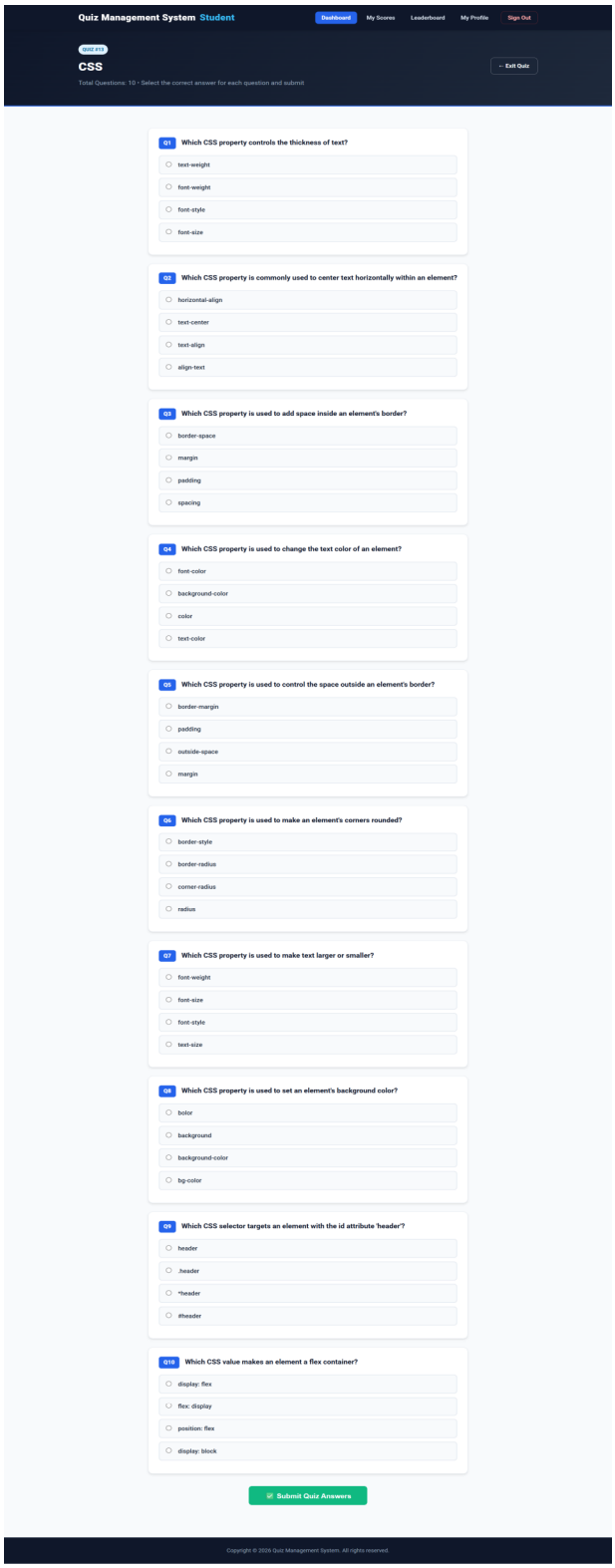


Figure 7.2.6: Student quiz attempt page

Source: takeq.php

```
<?php
$qs_sql = "SELECT * FROM questions WHERE quizid
= '$qid'";
$qs_res = mysqli_query($conn, $qs_sql);
$count = mysqli_num_rows($qs_res);

while ($row = mysqli_fetch_assoc($qs_res)) {
    $options = [trim($row['op1']), trim($row['op2']),
trim($row['op3'])];
    $correct_ans = trim($row['answer']);

    if (!in_array($correct_ans, $options)) {
        $options[] = $correct_ans;
    }
    shuffle($options);

    foreach ($options as $opt) {
```

```
        echo '<label><input type="radio" name="ans' . $i .
            '" value="' . htmlspecialchars($opt) . '" required>';
        echo htmlspecialchars($opt) . '</label>';
    }
}
<?php
$score = 0;
for ($i = 1; $i <= $count; $i++) {
    if (isset($_POST["ans" . $i], $post_answers[$i]) &&
        trim($_POST["ans" . $i]) ===
trim($post_answers[$i])) {
        $score++;
    }
}

$insert_sql = "INSERT INTO score (score, mail, quizid,
totalscore)
            VALUES ('$score', '$dbmail', '$qid', '$count')";
```

8.Testing

8.1 Testing Overview

Testing checks whether the application's features work as intended and whether the PHP pages interact correctly with the MySQL database. Testing for this project should cover registration, login, quiz management, question entry, quiz attempts, score recording, leaderboard display, and role-based access.

8.2 Testing Methodology

The application can be tested manually through a web browser with Apache and MySQL running in XAMPP. Use valid and invalid input values, complete the workflows as both staff and student users, and compare the observed behavior with the expected result. Check database records in phpMyAdmin where appropriate.

8.3 Unit Testing

Unit testing checks individual features separately. Examples include checking password confirmation during registration, login credential validation, quiz creation, question entry, score calculation, and profile display. Relevant pages include signup.php, index.php, homestaff.php, and takeq.php.

8.4 Integration Testing

Integration testing verifies that connected features work together. For example, confirm that a staff-created quiz appears on the student dashboard, that questions are retrieved for the selected quiz, and that submitted answers result in a score record linked to the correct student and quiz.

8.5 System Testing

System testing checks the complete application workflow. Test student and staff registration and login, role-appropriate dashboards, quiz creation and participation, score display, profile pages, leaderboard, and logout. Also check behavior for invalid credentials, missing form fields, quizzes without questions, and attempts to access pages without the correct login role.

8.6 Test Cases and Results

Run the following test cases and enter the observed result in the final column.

TC ID	Test Description	Expected & Actual Result	Status
TC-01	Submit valid student login credentials	Student is redirected to the student dashboard	PASS
TC-02	Submit valid staff/admin login credentials	Staff member is redirected to the administrative dashboard	PASS
TC-03	Submit an incorrect password	An appropriate authentication error message is displayed	PASS
TC-04	Register a new user with valid information	A new user account is created successfully	PASS
TC-05	Submit a form with missing required fields	Validation prevents submission and displays an error message	PASS
TC-06	Create a new quiz as a staff member	Quiz information is stored and the question-entry page is opened	PASS
TC-07	Add a multiple-choice question	Question, options, correct answer, and quiz association are stored	PASS
TC-08	Edit an existing quiz name	The selected quiz information is updated successfully	PASS

TC-09	Delete a quiz	The quiz is removed and associated questions are deleted	PASS
TC-10	Student views available quizzes	Available quizzes and related information are displayed	PASS
TC-11	Student submits correct and incorrect answers	The system calculates the score accurately	PASS
TC-12	Student views the completed quiz result	Score, total score, and performance remark are displayed	PASS
TC-13	Student views score history or leaderboard	Stored performance information is retrieved correctly	PASS
TC-14	Student attempts to access staff-only functionality	Access is restricted according to the user role	PASS
TC-15	User logs out	The session is ended and the user is returned to the login page	PASS

9.Conclusion and Future Scope

9.1 Conclusion and Future Scope

The **Quiz Management System** provides a web-based platform for staff to create and manage multiple-choice quizzes and for students to take quizzes and review their scores. It combines PHP application logic with a MySQL database and browser-based interfaces. Automatic answer evaluation and centralized result storage help reduce manual marking and keep quiz records accessible through the system.

The current version focuses on multiple-choice quizzes and core staff and student functions. Future work could extend the system with timed quizzes, randomized questions, additional question formats, expanded reporting, and other academic features.

9.2 Achievements

The project implements separate staff and student interfaces, user registration and authentication, quiz and question management, online quiz attempts, automatic score calculation, score history, and leaderboard views. It stores the main application data in MySQL and provides profile pages for users.

9.3 Limitations

The current system primarily supports multiple-choice questions. It does not provide features such as descriptive-answer evaluation, online proctoring, or a dedicated mobile application. Its deployment also depends on a configured PHP, Apache, and MySQL environment. These limitations define areas for future improvement.

9.4 Future Scope

Possible enhancements include adding timed quizzes, question and option randomization, additional question types, email notifications, password-reset support, more detailed performance reports, and stronger production deployment controls. A mobile application or integration with a learning-management system could also be considered.

10.Reference and Bibliography

10.1 Books and Academic References

1. Sommerville, Ian. *Software Engineering*. Pearson.
2. Elmasri, Ramez, and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson.
3. Pressman, Roger S., and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw Hill.
4. Welling, Luke, and Laura Thomson. *PHP and MySQL Web Development*. Addison-Wesley.

10.2 Official Documentation

- PHP Manual: https://www.php.net/docs.php
- MySQL Reference Manual: https://dev.mysql.com/doc/
- Apache HTTP Server Documentation: https://httpd.apache.org/docs/
- XAMPP: https://www.apachefriends.org/
- Bootstrap Documentation: https://getbootstrap.com/docs/
- AdminLTE Documentation: https://adminlte.io/docs/3.0/

10.3 Online Resources

The following online resources were consulted during the development and documentation of the Quiz Management System:

1. **PHP Manual** – Used for understanding PHP syntax, sessions, form handling, validation, file operations, and server-side programming.
   https://www.php.net/docs.php

2. **MySQL Documentation** – Used for database creation, SQL queries, relational tables, triggers, stored procedures, and database administration.
   https://dev.mysql.com/doc/

3. **Apache Friends XAMPP Documentation** – Used for configuring and operating the Apache, PHP, and MySQL development environment.
   https://www.apachefriends.org/

4. **MDN Web Docs** – Used for HTML5, CSS3, JavaScript, web forms, browser functionality, and client-server web concepts.
   https://developer.mozilla.org/

5. **Bootstrap Documentation** – Used for responsive layouts, forms, buttons, tables, cards, alerts, and other interface components.
   https://getbootstrap.com/docs/

6. **AdminLTE Documentation** – Used for dashboard layouts, sidebar navigation, administrative pages, cards, and interface styling.
   https://adminlte.io/docs/3.0/

7. **W3C Web Standards** – Used as a reference for web standards, HTML, CSS, accessibility, and interoperable web development.
   https://www.w3.org/standards/

These resources supported the implementation, testing, and documentation of the technologies used in the project.

Thank You